



PROJE TASARIM RAPORU

Ders: BİL 204 / YAZILIM MÜHENDİSLİĞİ

Bölüm/Şube: BİLGİSAYAR MÜHENDİSLİĞİ / 2.SINIF

Grup Adı: CodeSentinel

Proje Adı: Statik Kod Analizi ve Hata Tespit Sistemi

Teslim Tarihi: 04.05.2026

Grup Üyeleri:

- Ferhat Enes KOCABIYIK (24100011813)
- Hacı Ömer OĞUZ (24100011082)
- Hüseyin Mert KANLIDERE (24100011021)
- Kemal BABAOĞLAN (25100011472)
- Mehmet Enes İYİŞENYÜREK (23100011005)
- Mehmet UÇAR (24100011044)

İçindekiler

1. Giriş	4
1.1. Sistemin Amacı	4
1.2. Tasarım Hedefleri	4
1.2.1. Güvenilirlik (Reliability)	4
1.2.2. Performans (Performance)	4
1.2.3. Bakım Kolaylığı ve Genişletilebilirlik (Maintainability & Extensibility)	4
1.2.4. Kullanılabilirlik (Usability)	4
1.2.5. Doğruluk (Accuracy)	5
1.2.6. Otomasyon ve Kesintisiz Süreç Yönetimi (Automation)	5
1.2.7. İzlenebilirlik ve Kesin Hata Konumlandırma (Traceability)	5
1.2.8. Dış Aktarılabilirlik ve Birlikte Çalışabilirlik (Exportability & Interoperability)	5
1.2.9. Analiz Kapsamlılığı ve Güvenlik Odaklılık (Comprehensiveness & Security)	5
2. Üst Düzey Yazılım Mimarisi	6
2.1. Alt Sistem Ayırıştırması	6
2.2. Donanım/Yazılım Eşlemesi	6
2.2.1 Donanım Çalışma Ortamı ve Sistem Gereksinimleri	7
2.2.2 Programlama Dili, Paradigmalar ve Tasarım Kararları	7
2.2.3 İşletim Sistemi, Framework'ler ve Taşınabilirlik	7
2.3. Kalıcı Veri Yönetimi	8
2.3.1 Veritabanı Teknolojisi Seçimi ve Teknik Gereçekleri	8
2.3.2 Kalıcı Veri Formatları ve Çıktı Yönetimi	8
2.4. Erişim Kontrolü ve Güvenlik	11
2.5. Küresel Kontrol Akışı	12
2.6. Sınır Koşulları	12
2.6.1. Başlatma	12
2.6.2. Sonlandırma	12
2.6.3. Hata Durumları	13
3. Alt Düzey Tasarım	13
3.1. Nesne Tasarım Kararları	13
3.1.1. Birleşik Bulgu Modeli: Diagnostic Sınıfı	13
3.1.2. Kural Motoru: Strategy Pattern (IRule Arayüzü)	14
3.1.3. Raporlama: Strategy Pattern (IExporter Arayüzü)	14
3.1.4. Analiz Yönetimi: Facade Pattern (AnalysisEngine)	14
3.1.5. Mimari: Kapalı Katmanlı Mimari (Closed Architecture)	15
3.1.6. Veri Depolama: Dosya Sistemi vs. Veritabanı	15
3.2. Son Nesne Tasarımı	15
3.2.1. Uygulama Çekirdeği Paketi	15
3.2.2. Dosya Yönetimi Paketi	16
3.2.3. Ön İşleme Paketi	17
3.2.4. Sözdizimsel Analiz Paketi	17
3.2.5. Analiz Motorları Paketi	20
3.2.6. Sonuçlar ve Raporlama Paketi	22
3.2.7. Kullanıcı Arayüzü Paketi	24
3.2.8. Sınıflar Arası İlişkiler Özeti	26
3.3 Paketler / Katmanlar	26
3.3.1 Dış Paketler	27
3.3.2 İç Paketler	27
3.4 Sınıf Arayüzleri	29
3.4 Bölümü İçin Kısa Değerlendirme	45

3.5. Tasarım Desenleri.....	45
3.5.1. Strategy Pattern — IRule Denetimi.....	46
3.5.2. Strategy Pattern — IExporter (Raporlama).....	47
3.5.3. Facade Pattern — AnalysisEngine	48
Sözlük (Glossary)	50
Referanslar.....	51

1. Giriş

Bu bölümde projenin amacı ve temel tasarım hedefleri tanıtılır.

1.1. Sistemin Amacı

Sistemin temel amacı, C dilinde yazılmış kaynak kodları çalıştırmadan analiz edebilen, potansiyel hata ve güvenlik açıklarını tespit eden bir masaüstü statik kod analiz aracı geliştirmektir. Bu sistem, derleme aşamasında yakalanamayan bellek yönetimi problemlerini, tanımsız davranışlara yol açan yapıları ve mantıksal hataları erken aşamada tespit ederek yazılım geliştirme sürecindeki güvenilirliği artırmayı çözer. Sistemin hedef kullanıcıları arasında; C dili ile sistem/gömülü yazılım geliştiren yazılım geliştiriciler, hızlı geri bildirim almak isteyen sistem programcıları ve eğitim amacıyla analiz mantığını inceleyen bilgisayar mühendisliği öğrencileri yer almaktadır.

1.2. Tasarım Hedefleri

Projenin işlevsel olmayan (non-functional) gereksinimleri ve kalite hedefleri aşağıda açıklanmıştır:

1.2.1. Güvenilirlik (Reliability)

- Sistem, analiz esnasında oluşan hatalarda tamamen durmamalı ve çökmekten kaçınmalıdır.
- Ciddi sözdizimi (syntax) hataları oluşsa bile sistemin kısmi ayrıştırma yaparak (Kısmi AST oluşturarak) analiz sürecine devam etmesi ve sonucu kullanıcıya iletebilmesi sağlanacaktır.

1.2.2. Performans (Performance)

- Sistemin, orta büyüklükteki bir C kaynak kodunu kabul edilebilir bir süre içerisinde uçtan uca analiz edebilmesi hedeflenmektedir.
- İlerleyen aşamalarda performans artırımı sağlamak amacıyla çok çekirdekli işlem desteği ve çeşitli optimizasyon teknikleri sisteme entegre edilebilecektir.

1.2.3. Bakım Kolaylığı ve Genişletilebilirlik (Maintainability & Extensibility)

- Yazılım, nesne yönelimli programlama (OOP) prensiplerine uygun olarak, C++ diliyle modüler bir mimariye tasarlanacaktır.
- Analiz modülleri birbirinden bağımsız geliştirilebilir yapıda olacak ve dinamik kural motoru sayesinde sisteme yeni hata türü tespit kuralları kolayca eklenebilecektir.

1.2.4. Kullanılabilirlik (Usability)

- Sistem, bulguların ve metriklerin incelenmesini kolaylaştıran kullanıcı dostu bir grafiksel arayüze sahip olacaktır.
- Hata listesi üzerinden ilgili kod satırına otomatik navigasyon yapma, hataları önem derecelerine göre filtreleme ve editör üzerinde kod parçasını renk kodlarıyla vurgulama gibi işlevlerle deneyim zenginleştirilecektir.

1.2.5. Doğruluk (Accuracy)

- Yanlış pozitif (false positive) hata tespit oranlarını minimize etmek amacıyla kod kalitesi kuralları bağlam duyarlı olarak ve dikkatlice tasarlanacaktır.

1.2.6. Otomasyon ve Kesintisiz Süreç Yönetimi (Automation)

- **Bu Hedef Neden Önemli:** Analiz sürecinin kullanıcı müdahalesine gerek kalmadan baştan sona eksiksiz, tutarlı ve belirli bir sırayla otomatik olarak gerçekleşmesi zaman kazandırır ve kullanım kolaylığı sağlar.
- **Sistemde Nasıl Sağlanacak:** Sistemin merkezinde yer alan AnalysisEngine (Analiz Yöneticisi) orkestratörü sayesinde; kullanıcının arayüzden "Analiz Başlat" (UC-02) komutunu vermesiyle birlikte yorum ayıklama, kod ayrıştırma, syntax analizi, kural denetimi ve metrik hesaplama adımları art arda eş zamanlı (paralel) olarak otomatik çalıştırılacaktır.

1.2.7. İzlenebilirlik ve Kesin Hata Konumlandırma (Traceability)

- **Bu Hedef Neden Önemli:** Bir kod hatasının veya güvenlik açığının tespit edilmesi tek başına yeterli değildir; yazılım geliştiricinin sorunu hızla çözebilmesi için hatanın tam olarak hangi kod satırında (veya bloğunda) yer aldığını şeffaf bir şekilde görebilmesi gerekmektedir.
- **Sistemde Nasıl Sağlanacak:** Ön işleme modülü kaynak koddan yorum satırlarını ayıklarken orijinal satır numarası bütünlüğünü kesinlikle koruyacaktır. Ayrıştırma (Parser) aşamasında kodun soyut sözdizimi ağacı (AST) oluşturulurken veri akışı analiz edilerek pointerların adresleri takip edilecek; böylece tespit edilen hatalar doğrudan ilgili kod satırı vurgulanarak arayüzde konumlandırılacaktır.

1.2.8. Dışa Aktarılabilirlik ve Birlikte Çalışabilirlik (Exportability & Interoperability)

- **Bu Hedef Neden Önemli:** Statik analiz bulgularının yalnızca araç içerisinde anlık görüntülenmesi yerine; geliştirme ekiplerinin kalite geçmişini kayıt altına alabilmesi ve diğer inceleme araçlarına aktarabilmesi için standart formatlarda dışa aktarılabilmesi kritik bir gereksinimdir.
- **Sistemde Nasıl Sağlanacak:** Sonuçlar ve Raporlama modülü içerisinde yer alan Strateji (Strategy) tasarım deseni (IExporter arayüzü) kullanılarak, sistem analiz sonuçlarını kullanıcının isteğine göre yapılandırılmış formlarda olan TXT, JSON ve HTML formatlarında diske rapor olarak kaydedebilecektir (UC-12).

1.2.9. Analiz Kapsamlılığı ve Güvenlik Odaklılık (Comprehensiveness & Security)

- **Bu Hedef Neden Önemli:** C programlama dilinin esnek ve düşük seviyeli yapısı bellek yönetimi hatalarına ve güvenlik açıklarına (siber saldırı riski, sistem çökmeleri vb.) oldukça müsaittir. Bu nedenle sadece basit sözdizimi (syntax) değil, gizli sızıntıların da bulunması şarttır.
- **Sistemde Nasıl Sağlanacak:** Basit syntax kurallarının haricinde sistemin kural motoru; bellek sızıntıları, arabellek taşması (buffer overflow), dangling pointer (serbest bırakılmış bellek adresine yeniden erişim) kullanımını, çalıştırılması mümkün olmayan ölü kodları (dead code) ve başlatılmamış değişken kullanımını yakalayacak kadar derinlemesine statik analiz yapacaktır.

2. Üst Düzey Yazılım Mimarisi

Bu bölümde sistemin mimari yapısı, alt sistemlere ayrıştırma, donanım/yazılım eşlemesi, veri yönetimi, erişim kontrolü, kontrol akışı ve sınır koşulları ele alınır.

2.1. Alt Sistem Ayrıştırması

Statik Kod Analizi ve Hata Tespit Sistemi, bağımsız olarak test edilip geliştirilebilen 6 ana alt sistemden (modül/paket) oluşmaktadır. Bu alt sistemler ve aralarındaki ilişkiler aşağıda tanımlanmıştır:

- **Uygulama Çekirdeği (Application Core):** Sistemin giriş noktasını temsil eden ve yaşam döngüsünü yöneten ana katmandır. "AnalysisEngine" bileşeni aracılığıyla dosya yükleme işleminden başlayarak, analiz motorlarının çalıştırılmasına ve sonuçların arayüze iletilmesine kadar olan tüm orkestrasyonu koordine eder.
- **Dosya Yönetimi (File Management):** Sisteme yüklenen C kaynak kod dosyasının belleğe alınması, uzantı kontrolü (.c), boşluk kontrolü ve içerik okunabilirlik (validasyon) işlemlerinin yapıldığı alt sistemdir.
- **Ön İşleme ve Ayrıştırma (Preprocessing & Lexer/Parser):** Kaynak kodun öncelikle yorum satırlarından temizlendiği, ardından kelime gruplarına (token) ayrılarak sözlüksel analizin (Lexical Analysis) yapıldığı birimdir. Lexer verileri Parser'a aktarılır ve kodun yapısal temsilini ifade eden Soyut Sözdizimi Ağacı (AST- Abstract Syntax Tree) bu modülde üretilir.
- **Analiz Motorları (Analysis Engines):** Oluşturulan AST üzerinden derinlemesine incelemelerin yapıldığı ve paralel olarak çalışabilen iş mantığı katmanıdır. Bu alt sistem; sözdizimi hatalarını bulan SyntaxAnalyzer, bellek yönetimi ve kod kalitesini kurallara göre denetleyen RuleEngine ve sistem karmaşıklığı istatistiklerini (LOC, fonksiyon sayısı vb.) çıkartan MetricCalculator bileşenlerini barındırır.
- **Sonuçlar ve Raporlama (Results & Reporting):** Analiz motorlarından elde edilen bulguları derler ve SeverityClassifier yardımıyla "Kritik", "Uyarı" veya "Bilgi" seviyelerine göre sınıflandırır. Aynı zamanda bu sonuçların TXT, JSON veya HTML formatlarında dışa aktarılmasından (IExporter arayüzleri vasıtasıyla) sorumludur.
- **Kullanıcı Arayüzü (User Interface):** Kullanıcının sistemle doğrudan etkileşime girdiği dış katmandır. Kod editörü (CodeEditor), analiz sonuçlarının listelendiği panel (DiagnosticListPanel) ve istatistiklerin görüntülediği metrik paneli (MetricPanel) gibi sınır (boundary) bileşenlerinden oluşur.

2.2. Donanım/Yazılım Eşlemesi

Sistemin hangi donanım ve yazılım bileşenleri üzerinde çalışacağı ve bu bileşenlerin nasıl entegre edileceği aşağıda maddeler halinde belirtilmiştir.

2.2.1 Donanım Çalışma Ortamı ve Sistem Gereksinimleri

- **Düğüm Yapısı (Node Architecture):** Sistem, tek bir istemci düğümü (Client Node) üzerinde çalışan monolitik ancak modüler bir yapıya sahip masaüstü uygulamasıdır. Herhangi bir sunucu (Server) veya ağ (Network) bağlantısına ihtiyaç duymadan, tamamen kullanıcının yerel donanım kaynaklarını kullanır.
- **Minimum Sistem Gereksinimleri:** Statik analiz sürecinin (Lexical Analysis, Parsing) sistem kaynaklarını verimli kullanabilmesi için minimum donanım kısıtları şu şekildedir:

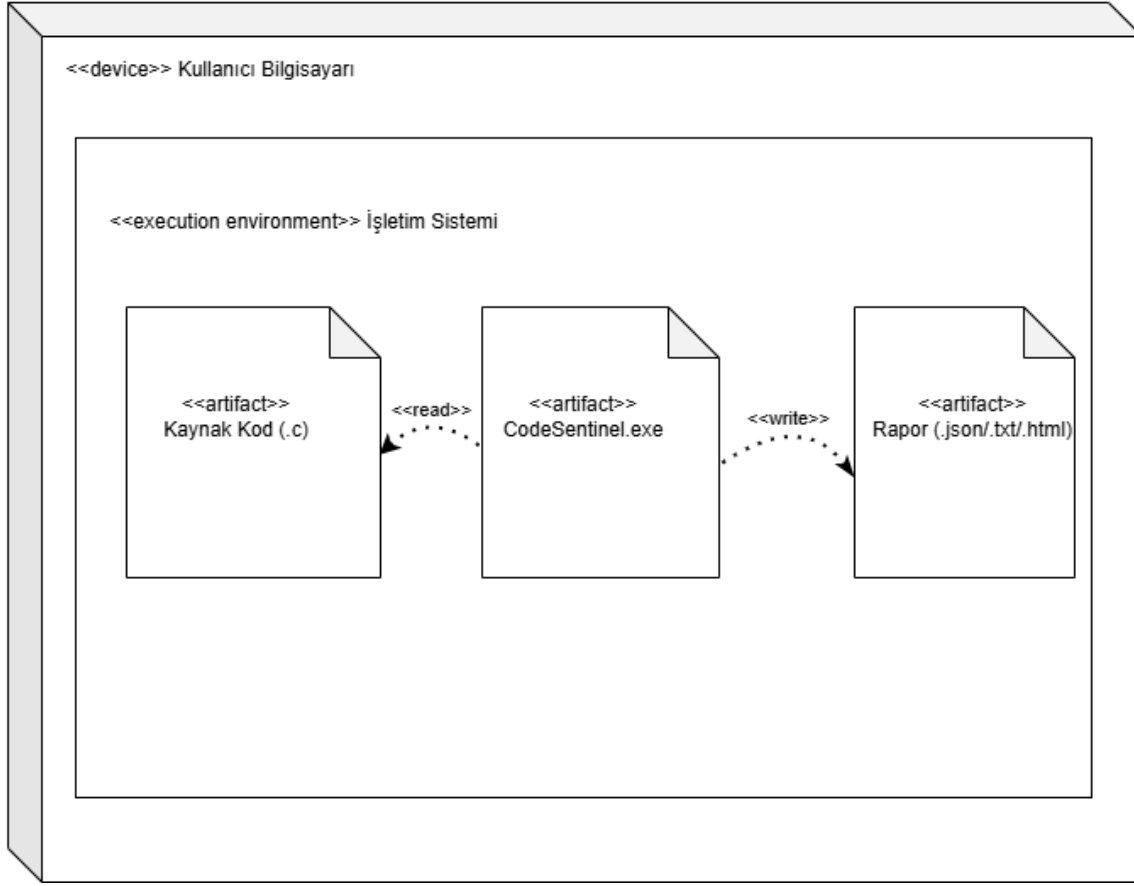
- **İşlemci (CPU):** Minimum 2.0 GHz Çift Çekirdekli (Dual-Core) işlemci. Kodun tokenlara ayrılması ve AST (Abstract Syntax Tree) üzerinde gezinme gibi işlemci yoğunluklu görevler yerel CPU üzerinde senkron yürütülecektir.
- **Bellek (RAM):** Minimum 4 GB RAM. Karmaşık .c dosyalarının analizi sırasında oluşan AST yapıları analiz süresince geçici olarak RAM üzerinde tutulacak, işlem bittiğinde temizlenecektir.
- **Depolama (Disk):** 100 MB boş alan. Dosya okuma ve üretilen kalıcı raporların diske yazılması işlemleri için gereklidir.
- **Giriş/Çıkış (I/O) Yönetimi:** Kaynak kodların okunması ve üretilen raporların kaydedilmesi işlemleri, işletim sisteminin dosya sistemi (File System) API'leri kullanılarak doğrudan yerel disk (SSD/HDD) üzerinde gerçekleştirilir.

2.2.2 Programlama Dili, Paradigmalar ve Tasarım Kararları

- **C++ Seçim Gerekçesi:** Statik analiz süreçleri yüksek miktarda metin işleme ve ağaç gezinme gerektirdiği için performans kritik bir hedeftir. Donanım seviyesine yakın kontrol ve yüksek çalışma zamanı verimliliği sunduğu için geliştirme dili olarak C++ tercih edilmiştir.
- **Nesne Yönelimli Tasarım (OOP):** Sistem, "Böl ve Yönet" ilkesine uygun olarak nesne yönelimli tasarlanmıştır. Kural analizi (RuleEngine) ve rapor dışı aktarma (ReportManager) modülleri, Strategy Pattern (Strateji Deseni) gibi OOP prensipleriyle kurgulanarak esneklik sağlanmıştır.
- **Kod Yönetimi:** Geliştirme sürecinin izlenebilirliği için GitHub üzerinden kamuya açık bir depo kullanılacaktır.

2.2.3 İşletim Sistemi, Framework'ler ve Taşınabilirlik

- **Çoklu Platform Desteği (Portability):** Uygulama; Windows, Linux ve macOS işletim sistemlerinde aynı kararlılıkla çalışacak şekilde tasarlanmıştır.
- **CMake Yapılandırması:** Sistemin platform bağımsız derlenebilmesi için CMake tercih edilmiştir.
- **Kütüphaneler:** Dışa bağımlılığı azaltmak ve sistemin sağlamlığını (robustness) artırmak için yalnızca C++ Standart Kütüphanesi (STL) bileşenleri kullanılacaktır.



ŞEKİL 2.2.1 Sistemin Yerleşim (Deployment) Diyagramı

2.3. Kalıcı Veri Yönetimi

Sistemin analiz sonucunda ürettiği verilerin kalıcı hale getirilmesi, veri tutarlılığı ve taşınabilirlik hedefleri doğrultusunda yapılandırılmıştır.

2.3.1 Veritabanı Teknolojisi Seçimi ve Teknik Gerekçeleri

Projede geleneksel bir ilişkisel veritabanı (RDBMS) yerine "**Dosya Tabanlı (File-based) Depolama**" teknolojisi tercih edilmiştir.

- **Performans ve Verimlilik:** Verilerin bir SQL sunucusuna bağlanıp tablo ilişkileriyle işlenmesi yerine, doğrudan disk üzerine yazılması yüksek performanslı bir çözümdür.
- **Taşınabilirlik (Portability):** Kullanıcının bilgisayarına ek bir veritabanı kurma zorunluluğu ortadan kaldırılarak "sıfır kurulum" (standalone) hedefi desteklenmiştir.
- **Basitlik (Simplicity):** Veritabanı bağlantı hataları ve şema yönetimi karmaşıklığından kaçınılmıştır.

2.3.2 Kalıcı Veri Formatları ve Çıktı Yönetimi

Analiz edilen veriler (AnalysisResult) ReportManager kontrol sınıfı tarafından yönetilir. "Strategy Pattern" kullanılarak dışa aktarma işlemi üç farklı sınıfa (TxtExporter, JsonExporter, HtmlExporter) bölünmüş ve veriler aşağıdaki tablodaki formatlarda diske kaydedilecek şekilde tasarlanmıştır (UC-12).

Format Türü	İhracatçı Sınıf	Kullanım Amacı ve İçerilen Veriler
.JSON	JsonExporter	Dış sistemlerle (CI/CD araçları, IDE'ler) entegrasyon için birincil veri yapısıdır. Analiz tarihi, dosya bilgisi, diagnostikler ve metrikleri hiyerarşik nesne dizileri (array) olarak saklar.
.TXT	TxtExporter	İnsan tarafından doğrudan komut satırında okunabilmesi için tasarlanmış düz metin log (kayıt) formatıdır. Sadece satır numarası ve hata/uyarı mesajlarını listeler.
.HTML	HtmlExporter	Analiz sonuçlarının tarayıcı üzerinde görsel olarak (renk kodlarıyla) sunulması için oluşturulan web tabanlı raporlama formatıdır.

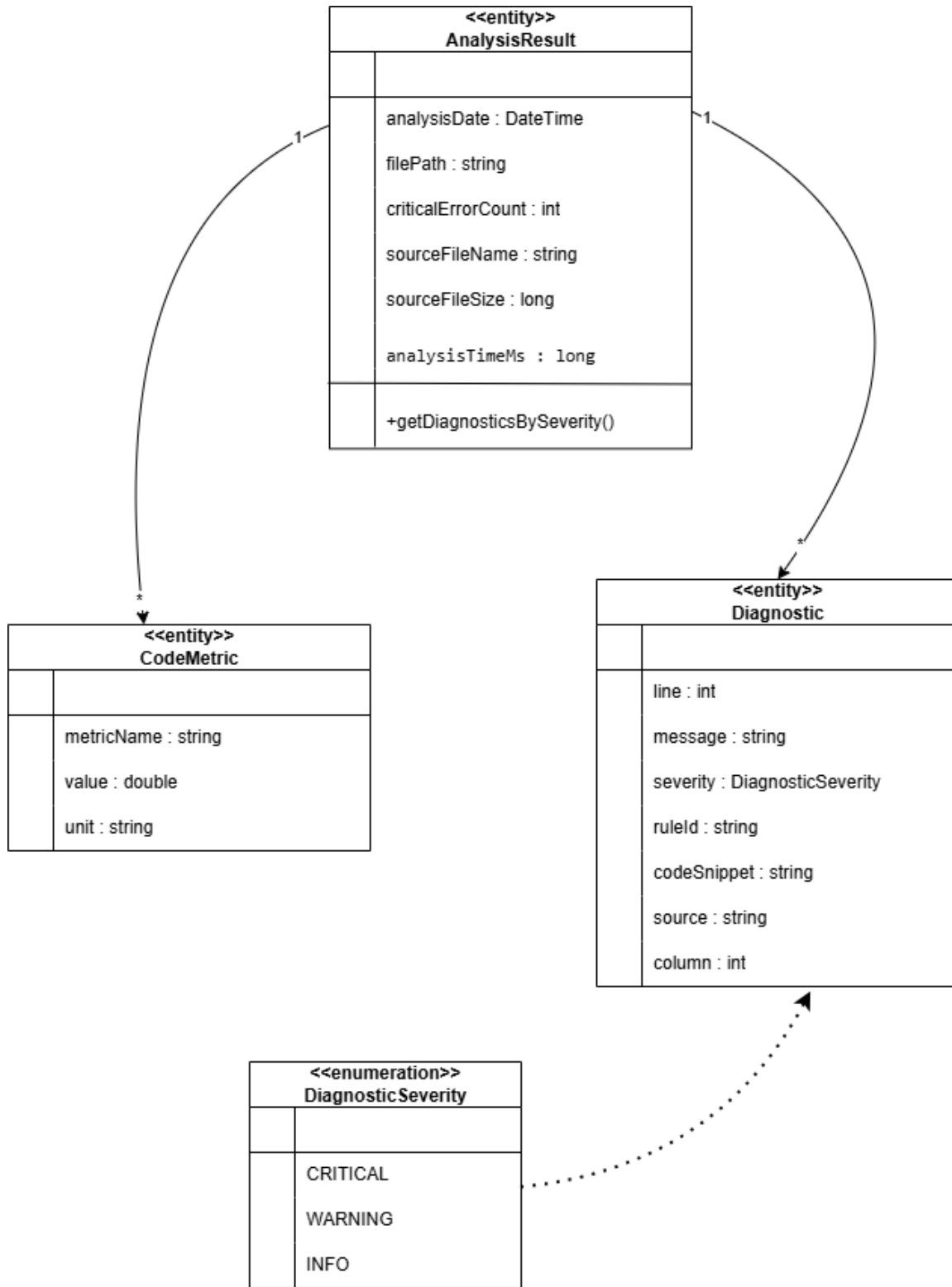
2.3.3 Veri Şeması ve Örnek JSON Yapısal Modeli

Sistemde fiziksel bir veritabanı bulunmadığı için standart bir ER diyagramı yerine, kalıcı verinin diske yazılma biçimini belirleyen yapısal şema ve örnek JSON çıktısı aşağıda modellenmiştir. Uygulamanın ürettiği AnalysisResult nesnesi, içerisinde Diagnostic ve CodeMetric dizilerini (array) barındırır.

```
{
  "analysisDate": "2026-05-04T15:30:00Z",
  "filePath": "src/main.c",
  "criticalErrorCount": 1,
  "metrics": [
    {
      "metricName": "LOC",
      "value": 120.0,
      "unit": "lines"
    },
    {
      "metricName": "FunctionCount",
      "value": 3.0,
      "unit": "count"
    }
  ],
  "diagnostics": [
    {
```

```
{
  "line": 14,
  "message": "Missing semicolon",
  "severity": "CRITICAL",
  "ruleId": "SYN001",
  "codeSnippet": "int x = 5"
},
{
  "line": 42,
  "message": "Unused variable 'temp'",
  "severity": "WARNING",
  "ruleId": "RUL002",
  "codeSnippet": "int temp;"
}
]
```

ŞEKİL 2.3.1 Örnek JSON Kalıcı Veri Çıktısı



ŞEKİL 2.3.2 JSON Veri Şeması ve Sınıf Modeli

2.4. Erişim Kontrolü ve Güvenlik

Sistem, yerel bir makinede çalışan bir masaüstü uygulaması olarak tasarlandığı için, web veya sunucu tabanlı sistemlerdeki gibi çoklu kullanıcı rolleri (Admin, Misafir vb.) ve kimlik doğrulama (login/şifre) mekanizmaları içermemektedir. Bu nedenle klasik bir erişim kontrol matrisine ihtiyaç

duyulmamaktadır. Sistemin tek ve birincil aktörü "Kullanıcı"dır ve sistemi başlatan her kullanıcı tüm işlevlere (Dosya Yükleme, Analiz Başlatma, Rapor Oluşturma) tam erişime sahiptir.

Güvenlik ve erişim kontrolü, işletim sistemi ve dosya sistemi seviyesinde şu şekilde sağlanmaktadır:

- **Veri Giriş Güvenliği (Validasyon):** Sisteme dışarıdan alınan dosyalar doğrudan işleme sokulmaz. FileValidator sınıfı aracılığıyla dosyanın .c uzantılı olup olmadığı, boş olup olmadığı ve okunabilir (metin formatında olup binary olmaması) durumu kontrol edilir. Desteklenmeyen veya bozuk dosyalar analiz motoruna ulaşmadan reddedilir.
- **Dosya Sistemi Erişim Kontrolü:** Analiz sonuçlarının rapor olarak diske kaydedilmesi aşamasında (UC-12), sistem işletim sisteminin dosya yazma izinlerini kontrol eder. Eğer ilgili dizinde yazma izni (Yetki Hatası) yoksa veya disk alanı yetersizse, sistem çökmez; "Yazma izni yok" veya "Yetersiz disk alanı" gibi uyarılarla kullanıcıyı bilgilendirerek işlemi iptal eder.

2.5. Küresel Kontrol Akışı

Statik Kod Analiz Aracı, olayları merkezi (centralized) bir kontrol mimarisi üzerinden yönetmektedir. Sistemin en önemli orkestrasyon noktası AnalysisEngine (Analiz Motoru) sınıfıdır. Kullanıcının arayüzden verdiği komutlar AnalysisEngine'e iletilir ve bu merkez, alt sistemleri belirli bir sırayla tetikler.

Kontrol akışı genel yapısı sıralı (senkron) ve eş zamanlı (paralel/asenkron) işlemlerin bir melezinden oluşmaktadır:

- **Sıralı Akış:** Dosya yükleme, yorum satırlarının ayıklanması (Ön İşleme) ve tokenizasyon ile Soyut Sözdizimi Ağacı (AST) oluşturulması işlemleri zorunlu olarak birbirini bekleyen ardışık adımlardır.
- **Eş Zamanlı (Paralel) Akış:** Performansı artırmak amacıyla tasarımda bir dallanma (fork) yapısı kurgulanmıştır. AST başarıyla (veya kısmi olarak) oluşturulduktan sonra; Syntax Hatası Tespiti, Kural Analizi Yapma ve Metrik Hesaplama alt modülleri eş zamanlı (paralel) olarak yürütülür.
- Paralel analizler tamamlandıktan sonra akış tekrar merkezde birleşir, SeverityClassifier (Uyarı Seviyesi Belirleyici) tüm bulguları derleyerek arayüze iletir.

2.6. Sınır Koşulları

Sistemin başlangıç, bitiş ve olağandışı (hata) durumlarındaki davranışları aşağıda açıklanmıştır:

2.6.1. Başlatma

- Uygulamanın giriş noktası olan Application kontrol sınıfı, ilk çalıştırmada MainWindow (Kullanıcı Arayüzü) ve AnalysisEngine nesnelerini oluşturarak sistemi başlatır.
- Sistem başlatıldığında herhangi bir veri yüklü değildir ve durum diyagramına göre başlangıçta "Boşta (Idle)" durumundadır.
- Arayüzdeki kod editörü ve metrik panelleri başlangıçta boştur; sistem kullanıcının geçerli bir .c kaynak kod dosyası seçmesini bekler.

2.6.2. Sonlandırma

- Uygulama sonlandırılırken kontrol yine Application sınıfındaki shutdown() metodu üzerinden sağlanır.
- Bu metot, analiz sırasında belleğe alınan SourceFile (Kaynak Dosya), token listeleri ve AST ağacı gibi verileri bellekten temizleyerek kaynakları işletim sistemine geri iade eder.

- Sistem kaynak kodu üzerinde deęişiklik yapmayan, sadece okuyan (statik analiz) bir araç olduęu için ani sonlandırmalarda orijinal kaynak kodun bozulması veya aktif oturum veri kaybı gibi riskler bulunmamaktadır.

2.6.3. Hata Durumları

Sistem "Güvenilirlik" tasarım hedefine uygun olarak hatalara karşı dayanıklı (resilient) şekilde tasarlanmıştır:

- Geçersiz Dosya Durumu: Kullanıcı .txt gibi geçersiz bir format, boş bir dosya veya okunamayan bir dizin yüklemeye çalışırsa sistem "Hata Durumu"na geçer, ilgili uyarı mesajını gösterir ve analizi hiç başlatmadan "Boşta" durumuna geri döner.
- Ayırıştırma (Parsing) Hataları: Kodun ayrıştırılması sırasında eksik noktalı virgül, eşleşmeyen parantez gibi kritik derleme hatalarıyla (Syntax Error) karşılaşılsa, sistem tamamen durmaz. Bunun yerine "Kısmi AST" üretir, hatayı kaydeder ve sonraki adımlara geçerek elindeki verilerle mümkün olan en fazla çıktıyı üretir.
- Kayıt Hataları: Rapor oluşturulurken diske yazma izni olmaması veya diskte yer kalmaması gibi beklenmeyen I/O (Giriş/Çıkış) hatalarında sistem "Rapor Hatası" durumuna geçer ve çokmeden kullanıcıya hatayı onaylatarak tekrar deneme fırsatı sunar.

3. Alt Düzey Tasarım

Bu bölümde sistemin sınıf düzeyindeki tasarımı, tasarım kararları, paketler, sınıf arayüzleri ve kullanılan tasarım desenleri detaylandırılır.

3.1. Nesne Tasarım Kararları

Bu bölümde tasarım sürecinde verilen kararlar ve bu kararların ardındaki gerekçeler açıklanmaktadır.

3.1.1. Birleşik Bulgu Modeli: Diagnostic Sınıfı

Alternatifler:

A) Ayrı sınıflar: SyntaxError ve Finding (kural ihlali) için iki ayrı sınıf tanımlamak.

B) Birleşik Diagnostic sınıfı: Her iki bulgu tipini tek sınıfta modellemek.

Tercih: B - Birleşik Diagnostic sınıfı

Gerekçe: Syntax hataları ve kural ihlalleri kullanıcı arayüzünde aynı listede gösterilmekte, aynı filtreleme işlemlerine tabi tutulmakta ve aynı raporlama akışından geçmektedir. Ayrı sınıf kullanmak, DiagnosticListPanel ve ReportManager içinde tekrarlayan kod yazılmasına ve iki farklı koleksiyonun ayrı ayrı yönetilmesine yol açardı. Diagnostic sınıfı source ve ruleId alanları aracılığıyla iki tip arasındaki ayrımı korumaktadır. Bu karar, arayüz ve raporlama katmanlarında kod tekrarını önemli ölçüde azaltmıştır.

3.1.2. Kural Motoru: Strategy Pattern (IRule Arayüzü)

Alternatifler:

A) Sabit kodlu if-else zinciri: RuleEngine içinde her kural için ayrı if bloęu.

B) Strategy Pattern: IRule arayüzü ve her kural için ayrı sınıf.

Tercih: B — Strategy Pattern

Gerekçe: Statik analiz araçlarının en kritik özelliği genişletilebilirliktir. Sabit if-else yaklaşımında yeni bir kural eklemek RuleEngine sınıfının değiştirilmesini gerektirirdi; bu Açık/Kapalı Prensiğini ihlal eder. IRule arayüzü sayesinde yeni bir kural getId(), getName(), getDefaultSeverity() ve check(ASTNode) metodlarını implement ederek sisteme eklenmekte, RuleEngine.addRule() ile çalışma zamanında kaydedilmektedir. Mevcut altı kural sınıfı (UnusedVariableRule, UninitializedVarRule, MagicNumberRule, FunctionLengthRule, GotoUsageRule, InfiniteLoopRule) bu deseni kullanmaktadır.

3.1.3. Raporlama: Strategy Pattern (IExporter Arayüzü)

Alternatifler:

A) Tek ReportManager sınıfı içinde format kontrolü: if format == "txt" / if format == "json" dallanması.

B) Strategy Pattern: IExporter arayüzü ve her format için ayrı sınıf.

Tercih: B — IExporter arayüzü

Gerekçe: Analiz sonuçlarının TXT, JSON ve HTML formatlarında kaydedilebilmesi gerekmektedir. Tek sınıf yaklaşımı, yeni format (PDF, CSV vb.) eklendikçe ReportManager'ın büyümesine ve çok sayıda if-else bloğu içermesine neden olurdu. IExporter arayüzü, her formatın kendi export() implementasyonuna sahip olmasını sağlarken ReportManager'ı değişmeden tutar. TxtExporter, JsonExporter ve HtmlExporter sınıfları bu arayüzü implement etmektedir.

3.1.4. Analiz Yönetimi: Facade Pattern (AnalysisEngine)

Alternatifler:

A) Dağıtık kontrol: Kullanıcı arayüzünün Preprocessor, Lexer, Parser vb. sınıfları doğrudan çağırması.

B) Facade Pattern: AnalysisEngine sınıfının tüm analiz pipeline'ını yönetmesi.

Tercih: B — Facade Pattern

Gerekçe: Analiz süreci sekiz adımdan oluşmaktadır: ön işleme, tokenization, ayrıştırma, syntax analizi, kural analizi, metrik hesaplama, önem seviyesi sınıflandırma ve sonuç derleme. Bu karmaşıklığın kullanıcı arayüzüne sızdırılması, UI katmanının analiz mantığına sıkı sıkıya bağlanmasına yol açardı. AnalysisEngine, yalnızca analyze(SourceFile) çağrısıyla tüm pipeline'ı sırasıyla koordine ederek tek bir AnalysisResult döndürmektedir. Bu yaklaşım, arayüz katmanını analiz değişikliklerinden soyutlamaktadır.

3.1.5. Mimari: Kapalı Katmanlı Mimari (Closed Architecture)

Alternatifler:

A) Açık mimari: Her katmanın herhangi bir alt katmana doğrudan erişmesi.

B) Kapalı mimari: Her katmanın yalnızca bir altındaki katmana erişmesi.

Tercih: B — Kapalı Katmanlı Mimari

Gerekçe: Üç katmanlı mimaride (UI → Uygulama → Dosya) UI katmanının doğrudan FileLoader'a erişmesi, bağımlılıkları artırır ve değişikliklerin yayılma riskini yükseltir. Kapalı mimaride

MainWindow, yalnızca AnalysisEngine ile konuşur; AnalysisEngine ise dosya işlemleri için FileLoader'ı kullanır. Bu yaklaşım alt sistemler arası bağımlılığı azaltmakta, bakım yapılabilirliği artırmaktadır.

3.1.6. Veri Depolama: Dosya Sistemi vs. Veritabanı

Alternatifler:

A) SQLite veritabanı: Analiz geçmişini ve sonuçlarını kalıcı olarak saklamak.

B) Dosya sistemi: Kullanıcı tarafından talep edildiğinde TXT/JSON/HTML dosyasına yazma.

Tercih: B — Dosya sistemi

Gerekçe: Sistem masaüstü, tek kullanıcılı bir statik analiz aracıdır. Analiz geçmişi tutma veya çok kullanıcılı paylaşım gereksinimleri bulunmamaktadır. Veritabanı kullanımı ek bağımlılık ve kurulum karmaşıklığı getirir; dosya tabanlı dışa aktarma kullanıcının beklentilerini karşılamaktadır.

3.2. Son Nesne Tasarımı

Bu bölümde sistemin tüm sınıflarını, özniteliklerini, metotlarını ve aralarındaki ilişkileri gösteren ayrıntılı UML sınıf tasarımı açıklanmaktadır. Tasarım, analiz raporundaki sınıf modelini temel alarak tasarım kararları doğrultusunda genişletilmiş ve detaylandırılmıştır.

3.2.1. Uygulama Çekirdeği Paketi

Application «control» Uygulamanın giriş noktasını ve yaşam döngüsünü yöneten üst düzey sınıftır.

Öznitelikler	- engine : AnalysisEngine, window : MainWindow
Metotlar	+ run() : void — MainWindow ve AnalysisEngine'i başlatır, uygulamayı çalıştırır. + shutdown() : void — Kaynakları serbest bırakarak uygulamayı sonlandırır.

AnalysisEngine «control» Tüm analiz pipeline'ının merkezi yürütücüsüdür.

Öznitelikler	- fileLoader : FileLoader, preprocessor : Preprocessor, lexer : Lexer, - parser : Parser, - syntaxAnalyzer : SyntaxAnalyzer, - ruleEngine : RuleEngine, - metricCalculator : MetricCalculator, severityClassifier : SeverityClassifier
Metotlar	+ analyze(file : SourceFile) : AnalysisResult — Tüm analiz adımlarını sırasıyla yürütür; tek AnalysisResult döner. + isReady() : bool — Tüm bileşenler hazır mı denetler.

3.2.2. Dosya Yönetimi Paketi

SourceFile «entity» Yüklene C kaynak kod dosyasını temsil eden varlık nesnesidir.

Öznitelikler	- filePath : string, - fileName : string, - content : string, fileSize : long, - loadDate : DateTime
Metotlar	+ getContent() : string, + getFileName() : string, + getFilePath() : string, + getFileSize() : long, + isEmpty() : bool

FileLoader «control» diskten dosya okuyarak SourceFile nesnesi oluşturur.

Öznitelikler	—
Metotlar	+ load(path : string) : SourceFile — Dosyayı okur, FileValidator'dan geçirir, SourceFile döner. - readFromDisk(path : string) : string — Ham dosya içeriğini okur.

FileValidator «control» Dört aşamalı doğrulama yapar.

Öznitelikler	—
Metotlar	+ validate(file : SourceFile) : ValidationResult — Tüm kontrolleri çalıştırır. checkExtension(path : string) : bool — .c uzantısı kontrolü. checkNotEmpty(content : string) : bool — Boşluk kontrolü. checkReadable(path : string) : bool — Okunabilirlik kontrolü. checkNotBinary(content : string) : bool — Binary dosya kontrolü.

ValidationResult «entity» Doğrulama sonucunu taşıyan hafif veri nesnesidir.

Öznitelikler	- valid : bool, - errorMessage : string
Metotlar	+ isValid() : bool, + getErrorMessage() : string

3.2.3. Ön İşleme Paketi

Preprocessor «control» Yorum satırlarını ayıklayarak temizlenmiş kodu üretir.

Öznitelikler	- rawCode : string, - cleanedCode : string, commentCount : int + process(code : string) : string
Metotlar	getCommentLineCount() : int removeSingleLineComments(code : string) : string removeMultiLineComments(code : string) : string isInsideStringLiteral(code : string, pos : int) : bool

3.2.4. Sözdizimsel Analiz Paketi

Lexer «control» Temizlenmiş kaynak kodu tokenlara ayırır.

Öznitelikler	- source : string, - position : int, - line : int, - column : int
Metotlar	+ tokenize(code : string) : Token[]- advance() : char, peek() : char readKeywordOrIdentifier() : Token, - readNumericLiteral() : Token- readStringLiteral() : Token, - readOperator() : Token, - skipWhitespace() : void

Token «entity» Sözlüksel analizde üretilen tek bir kod birimini temsil eder.

Öznitelikler	- type : TokenType, - value : string, - line : int, - column : int
Metotlar	+ getType() : TokenType, + getValue() : string, + getLine() : int, + getColumn() : int

TokenType «enum» : KEYWORD, IDENTIFIER, INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL, CHAR_LITERAL, OPERATOR, PUNCTUATION, PREPROCESSOR, UNKNOWN

Parser «control» Token dizisini alarak AST oluşturur.

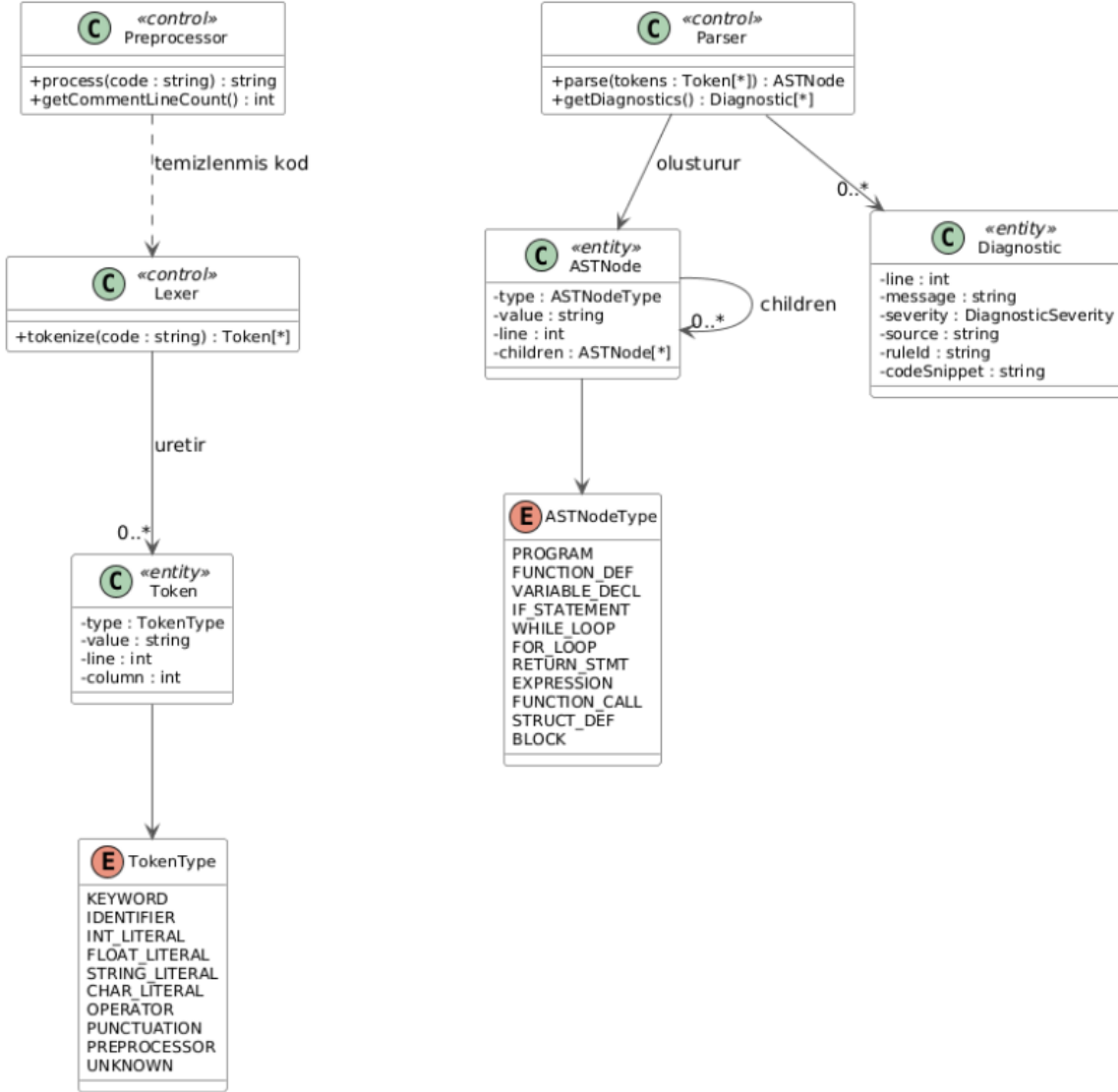
Öznitelikler	- tokens : Token[], - pos : int, - root : ASTNode, - diagnostics : Diagnostic[]
Metotlar	+ parse(tokens : Token[]) : ASTNode, + getDiagnostics() : Diagnostic[] - parseProgram() : ASTNode, - parseFunctionDef() : ASTNode, - parseBlock() : ASTNode - parseStatement() : ASTNode, - parseExpression() : ASTNode - parseIfStatement() : ASTNode, - parseWhileLoop() : ASTNode - parseForLoop() : ASTNode, - parseStructDef() : ASTNode - expect(type : TokenType) : Token, - match(type : TokenType) : bool

ASTNode «entity» soyut Sözdizim Ağacı'ndaki tek bir düğümü temsil eder.

Öznitelikler	- type : ASTNodeType, - value : string, - line : int, - children : ASTNode[], - parent : ASTNode
Metotlar	- addChild(child : ASTNode) : void, + getChildren() : ASTNode[] + getType() : ASTNodeType, + getValue() : string, + getLine() : int, + getParent() : ASTNode

ASTNodeType «enum»: PROGRAM, FUNCTION_DEF, VARIABLE_DECL, IF_STATEMENT, WHILE_LOOP, FOR_LOOP, RETURN_STMT, ASSIGNMENT, EXPRESSION, BINARY_OP, FUNCTION_CALL, STRUCT_DEF, BLOCK

On Isleme, Lexer & Parser Modulu



3.2.5. Analiz Motorları Paketi

SyntaxAnalyzer «control» AST ve token listesi üzerinde syntax hatalarını tespit eder.

Öznitelikler	—
Metotlar	<code>+ analyze(ast : ASTNode, tokens : Token[]) :</code> <code>Diagnostic[] checkMissingSemicolon(node : ASTNode) :</code> <code>void checkUnmatchedBrackets(tokens : Token[]) :</code> <code>void checkUnmatchedParentheses(tokens :</code> <code>Token[]) :</code> <code>void checkInvalidDeclarations(node :</code> <code>ASTNode) : void</code>

RuleEngine «control» tüm IRule nesnelerini yöneterek AST üzerinde sırasıyla çalıştırır.

Öznitelikle r	- rules : IRule[]
Metotlar	+ addRule(rule : IRule) : void, + removeRule(ruleId : string) : void+ analyzeAll(ast : ASTNode) : Diagnostic[], + getRuleCount() : int

IRule «interface» — Strategy Pattern

Metotlar: + getId() : string, + getName() : string+ getDefaultSeverity() : DiagnosticSeverity, + check(ast : ASTNode) : Diagnostic[]

UnusedVariableRule «entity» tanımlanmış ancak hiç kullanılmamış değişkenleri tespit eder.
+ check(ast : ASTNode) : Diagnostic[] - collectDeclaredVars(node : ASTNode) : string[]-
collectUsedVars(node : ASTNode) : string[]

UninitializedVarRule «entity» ilk değer atanmadan kullanılan değişkenleri tespit eder (tanımsız davranış riski).
+ check(ast : ASTNode) : Diagnostic[]

MagicNumberRule «entity» 0 ve 1 dışındaki açıklamasız sayısal sabitleri tespit eder.
-allowedValues : int[] (varsayılan: {0, 1}) + check(ast : ASTNode) : Diagnostic[]

FunctionLengthRule «entity» Yapılandırılabilir maksimum satır sayısını (varsayılan: 50) aşan fonksiyonları tespit eder.
-maxLines : int = 50 + check(ast : ASTNode) : Diagnostic[],+ setMaxLines(max : int) : void

GotoUsageRule «entity» goto ifadesi kullanımını tespit eder; yapısal programlama prensiplerine aykırıdır.
+ check(ast : ASTNode) : Diagnostic[]

InfiniteLoopRule «entity» Potansiyel sonsuz döngü yapılarını (örn: güncellenmemeyen döngü değişkeni) tespit eder.
+ check(ast : ASTNode) : Diagnostic[]

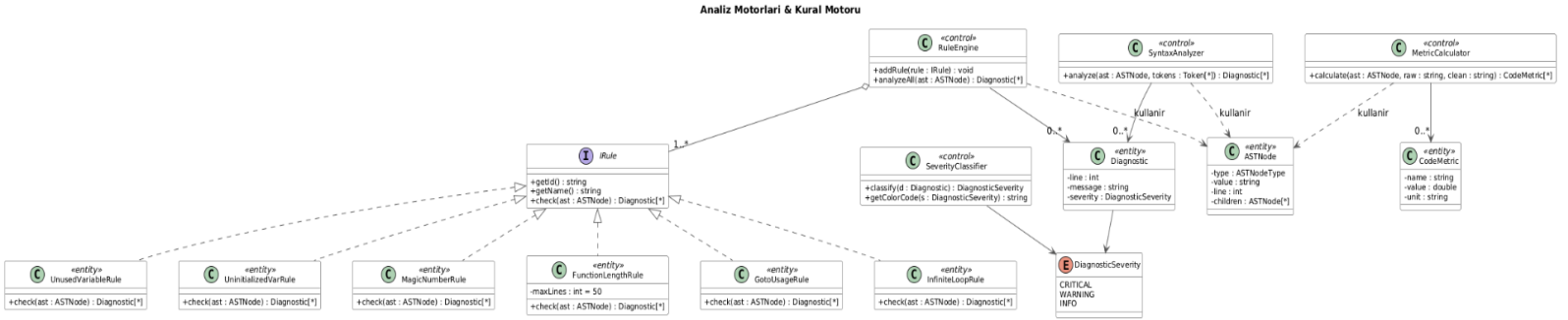
MetricCalculator «control» AST ve kaynak kod üzerinde nicel kod metriklerini hesaplar.

Metotlar	+ calculate(ast : ASTNode, rawCode : string, cleanCode : string) : CodeMetric[]countTotalLines(code : string) : int- countCodeLines(code : string) : intcountEmptyLines(code : string) : int- countCommentLines(raw : string, clean : string) :
-----------------	---

```

intcountFunctions(ast : ASTNode) : int-
countStructs(ast : ASTNode) :
intcountVariables(ast : ASTNode) : int-
averageFunctionLength(ast : ASTNode) :
double

```



3.2.6. Sonuçlar ve Raporlama Paketi

DiagnosticSeverity «enum» CRITICAL (Kırmızı — derleme hatası), WARNING (Sarı — potansiyel sorun), INFO (Mavi — bilgilendirme)

Diagnostic «entity» Syntax hatası ve kural ihlalini tek sınıfta birleştiren birleşik bulgu nesnesidir.

Öznitelikler	- line : int, - column : int, - message : string, - severity : DiagnosticSeverity - source : string (syntax / rule), - ruleId : string, - codeSnippet : string
Metotlar	+ getLine() : int, + getMessage() : string, + getSeverity() : DiagnosticSeverity + getSource() : string, + getRuleId() : string, + getCodeSnippet() : string

CodeMetric «entity» hesaplanan tek bir kod metriğini temsil eden veri nesnesidir.

Öznitelikler	- name : string, - value : double, - unit : string, - description : string
Metotlar	+ getName() : string, + getValue() : double, + getUnit() : string, + getDescription() : string

SeverityClassifier «control» her bulguya uygun önem seviyesi atayan ve renk kodu belirleyen sınıftır.

Metotlar	+ classify(diagnostic : Diagnostic) : DiagnosticSeverity + getColorCode(severity : DiagnosticSeverity) : string + getLabel(severity : DiagnosticSeverity) : string
-----------------	--

AnalysisResult «entity» Bir analiz çalışmasının tüm çıktılarını birleştiren bütünleyici veri sınıfıdır.

Öznitelikler	- sourceFileName : string, - sourceFileSize : long, - analysisDate : DateTime - diagnostics : Diagnostic[], - metrics : CodeMetric[], - analysisTimeMs : long
Metotlar	+ getDiagnostics() : Diagnostic[], + getMetrics() : CodeMetric[] + getDiagnosticsBySeverity(s : DiagnosticSeverity) : Diagnostic[] + getCriticalCount() : int, + getWarningCount() : int + getInfoCount() : int, + getTotalDiagnosticCount() : int, + hasErrors() : bool

IExporter «interface» — Strategy Pattern

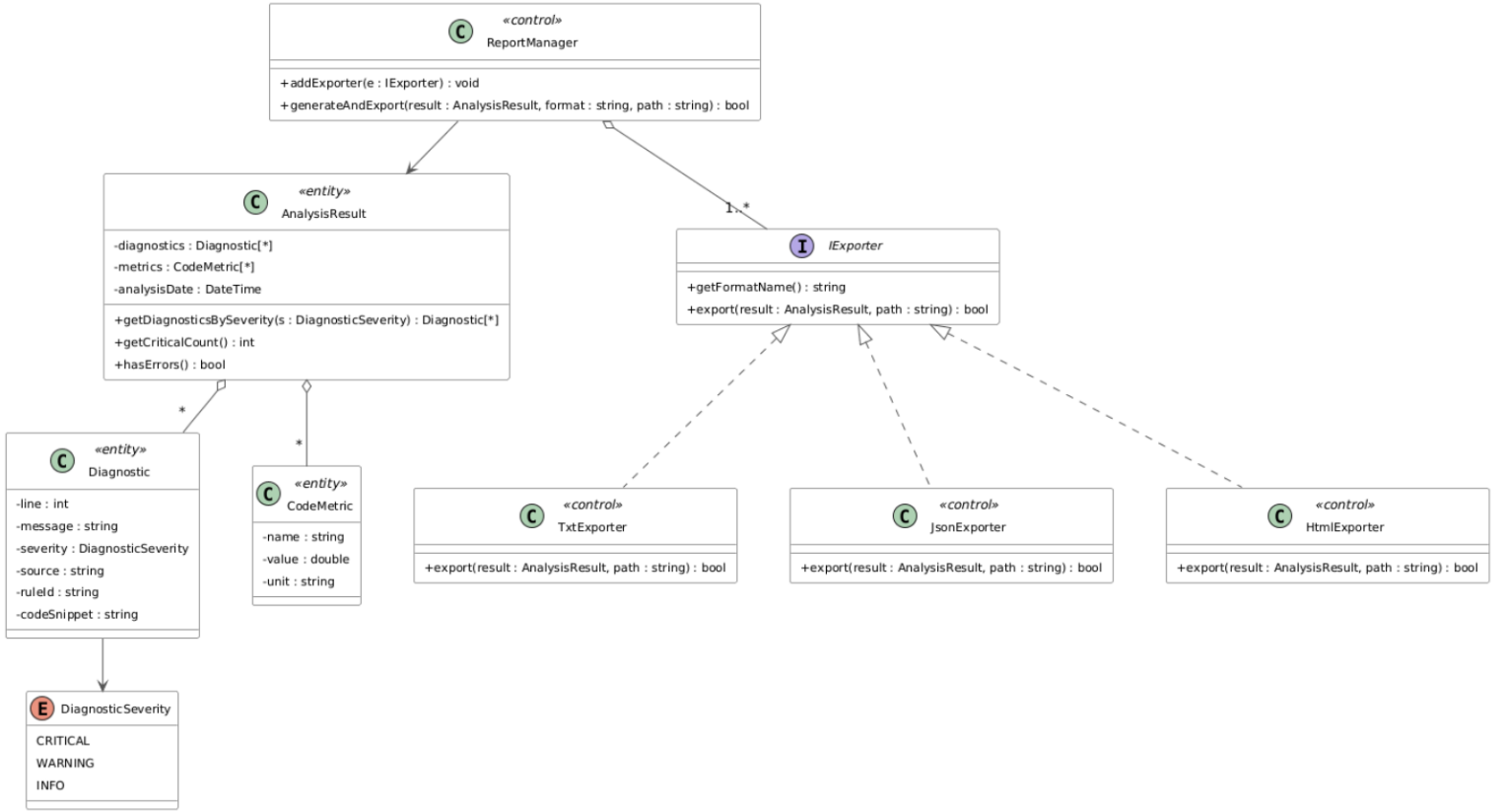
Metotlar	+ getFormatName() : string, + getFileExtension() : string + export(result : AnalysisResult, path : string) : bool
-----------------	---

TxtExporter / JsonExporter / HtmlExporter «control» IExporter arayüzünü uygulayan somut dışa aktarma sınıflarıdır. Her biri getFormatName(), getFileExtension() ve export(result, path) metodlarını kendi formatına özgü biçimde implement eder.

ReportManager «control» Dışa aktarma sürecini yöneten koordinatör sınıftır.

Öznitelikler	- exporters : IExporter[]
Metotlar	+ addExporter(exports : IExporter) : void + generateAndExport(result : AnalysisResult, format : string, path : string) : bool + getAvailableFormats() : string[]

Sonuçlar & Raporlama Modülü



3.2.7. Kullanıcı Arayüzü Paketi

MainWindow «boundary» uygulamanın ana penceresidir; tüm UI bileşenlerini barındırır.

Öznitelikler	- codeEditor : CodeEditor, - diagnosticPanel : DiagnosticListPanel - metricPanel : MetricPanel, - toolbar : Toolbar, - statusBar : StatusBar
Metotlar	+ initialize() : void, + showResults(result : AnalysisResult) : void, + showError(message : string) : void

CodeEditor «boundary» kaynak kodun gösterildiği ve navigasyon yapıldığı editör alanıdır.

+ loadContent(code : string) : void, + goToLine(line : int) : void + highlightLines(start : int, end : int, color : string) : void, + clearAllHighlights() : void

DiagnosticListPanel «boundary» analiz bulgularını listeleyen ve filtreleme imkânı sunan bileşendir.

Metotlar:

+ displayDiagnostics(d : Diagnostic[]) : void, + filterBySeverity(s : DiagnosticSeverity) : void + clearFilters() : void, + onItemClick(item : Diagnostic) : void

MetricPanel «boundary» Hesaplanan kod metriklerini özet biçimde gösterir.

Metotlar:

+ displayMetrics(m : CodeMetric[]) : void

ToolBar «boundary» üst düzey işlem butonlarını barındırır (UC-01, UC-02, UC-11).

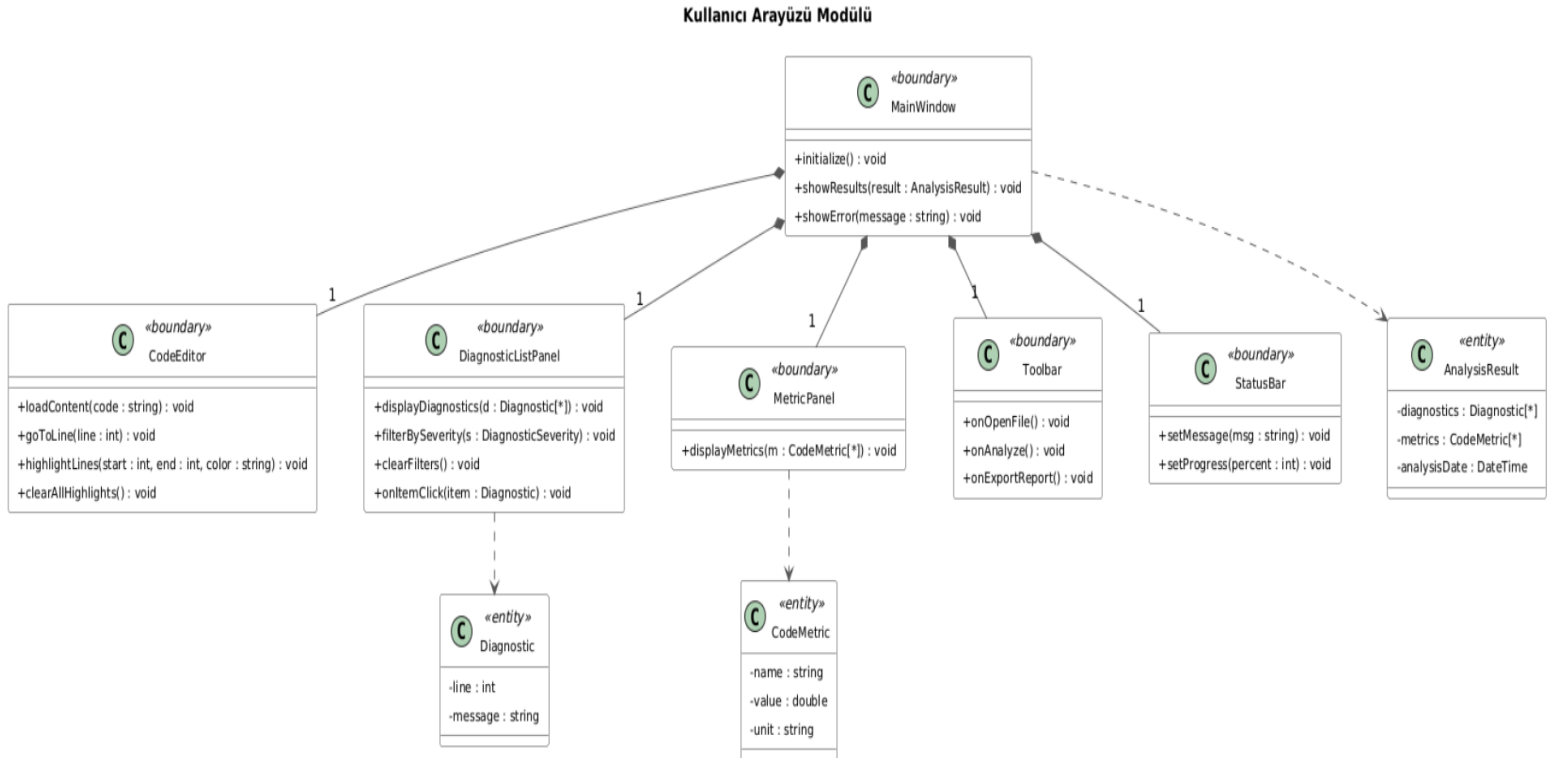
Metotlar:

+ onOpenFile() : void, + onAnalyze() : void, + onExportReport() : void

StatusBar «boundary» anlık durum mesajları ve analiz ilerleme bilgisi gösterir.

Metotlar:

+ setMessage(msg : string) : void, + setProgress(percent : int) : void



3.2.8. Sınıflar Arası İlişkiler Özeti

İlişki	Tip	Açıklama
Application → MainWindow	Kompozisyon (1-1)	Uygulama ana pencereye sahiptir
Application → AnalysisEngine	Kompozisyon (1-1)	Uygulama analiz motoruna sahiptir
AnalysisEngine → [tüm analiz bileşenleri]	Kompozisyon	Engine tüm alt bileşenlere sahiptir
AnalysisEngine → AnalysisResult	Bağımlılık	Engine sonuç üretir
RuleEngine → IRule	Agregasyon (1..*)	Engine birden fazla kural barındırır
IRule ← UnusedVariableRule vb.	Gerçekleştirim	Her kural IRule’u implement eder
ReportManager → IExporter	Agregasyon (1..*)	Manager birden fazla exporter barındırır
IExporter ← TxtExporter vb.	Gerçekleştirim	Her exporter IExporter’ı implement eder
AnalysisResult → Diagnostic	Agregasyon (*)	Sonuç birden fazla bulgu içerir
AnalysisResult → CodeMetric	Agregasyon (*)	Sonuç birden fazla metrik içerir
MainWindow → [UI bileşenleri]	Kompozisyon	Ana pencere tüm panel bileşenlerine sahiptir
DiagnosticListPanel → Diagnostic	Bağımlılık	Panel bulguları görüntüler
MetricPanel → CodeMetric	Bağımlılık	Panel metrikleri görüntüler

3.3 Paketler / Katmanlar

CodeSentinel, C dilinde yazılmış kaynak kodları çalıştırmadan analiz eden masaüstü tabanlı bir statik kod analiz aracıdır. Sistem; dosya yükleme, doğrulama, ön işleme, tokenization, AST oluşturma, syntax analizi, kural analizi, metrik hesaplama, sonuç gösterimi ve raporlama adımlarından oluşur. Bu nedenle proje paketleri, her modülün sorumluluğu net olacak şekilde ayrılmıştır.

Paket yapısında temel amaç; kullanıcı arayüzü, analiz mantığı, veri modeli ve raporlama bileşenlerinin birbirinden ayrılmasıdır. Bu ayrım sayesinde yeni bir analiz kuralı, yeni bir rapor formatı veya yeni bir arayüz bileşeni eklenirken mevcut sınıfların büyük kısmının değiştirilmesine gerek kalmaz.

3.3.1 Dış Paketler

Paket / Kütüphane	Kullanım Yeri	Rol / Gerekçe
C++ Standard Library (<string>, <vector>, <map>, <memory>)	Tüm katmanlar	Metin işleme, koleksiyon tutma, nesne yönetimi ve temel algoritmalar için kullanılır.
C++ <filesystem>	file, reporting	Kaynak dosyanın yolu, adı, uzantısı ve rapor çıktılarının kayıt konumu yönetilir.
Qt Framework	ui	MainWindow, CodeEditor, DiagnosticListPanel, MetricPanel, Toolbar ve StatusBar gibi masaüstü arayüz bileşenleri geliştirilir.
CMake	Proje yapılandırması	C++ kaynak dosyalarının platform bağımsız şekilde derlenmesi ve bağımlılıkların düzenlenmesi sağlanır.
Git / GitHub	Ekip geliştirme süreci	Sürüm kontrolü, ekip içi kod paylaşımı ve değişiklik takibi için kullanılır.

Dış paketler, sistemin geliştirme ve çalışma ortamını destekler. Statik analiz mantığı, proje içinde tanımlanan iç paketlerde yer alır.

3.3.2 İç Paketler

İç paketler aşağıdaki tabloda; içerdiği sınıflar, sorumlulukları ve diğer paketlerle ilişkileriyle birlikte verilmiştir.

Paket	İçerdiği Sınıflar	Sorumluluk	İlişkili Paketler
core	Application, AnalysisEngine	Uygulama yaşam döngüsünü ve analiz akışını yönetir. AnalysisEngine, analiz sürecinin merkezi orkestratördür.	file, preprocessing, parsing, analysis, results, ui
file	FileLoader, FileValidator, SourceFile, ValidationResult	C kaynak dosyasını yükler, .c uzantısı ve okunabilirlik gibi kontrolleri yapar, dosya bilgisini SourceFile nesnesinde tutar.	core, results
preprocessing	Preprocessor	Tek satırlı ve çok satırlı yorumları analiz dışı bırakır; satır numarası bütünlüğünü korur.	core, parsing

parsing	Lexer, Token, TokenType, Parser, ASTNode, ASTNodeType	Kaynak kodu tokenlara ayırır ve AST veri yapısını oluşturur.	preprocessing, analysis, results
analysis	SyntaxAnalyzer, RuleEngine, IRule, UnusedVariableRule, UninitializedVarRule, MagicNumberRule, FunctionLengthRule, GotoUsageRule, InfiniteLoopRule, MetricCalculator	Syntax hatalarını, kod kalitesi ihlallerini ve kod metriklerini üretir.	parsing, results
results	Diagnostic, DiagnosticSeverity, CodeMetric, SeverityClassifier, AnalysisResult	Hata, uyarı, bilgi bulguları ve metrik sonuçlarını ortak veri modellerinde tutar.	analysis, reporting, ui
reporting	ReportManager, IExporter, TxtExporter, JsonExporter, HtmlExporter	Analiz sonucunu rapora dönüştürür ve TXT, JSON, HTML formatlarında dışa aktarır.	results, file
ui	MainWindow, CodeEditor, DiagnosticListPanel, MetricPanel, Toolbar, StatusBar	Kullanıcı etkileşimini, kod görüntülemeyi, bulgu listelemeyi, filtrelemeyi ve navigasyonu sağlar.	core, results, reporting

core Paketi

Application sınıfı uygulamanın başlangıç ve kapanış sürecini yönetir. AnalysisEngine sınıfı ise dosya yüklendikten sonra tüm analiz zincirini tek noktadan yürütür. Bu paket, kullanıcı arayüzünün analiz alt bileşenlerini doğrudan bilmesini engeller.

file Paketi

FileLoader fiziksel dosyayı okur; FileValidator dosyanın uzantı, boşluk ve okunabilirlik kontrollerini yapar. SourceFile, dosya adı, yol, içerik ve boyut gibi bilgileri taşır. ValidationResult ise doğrulama sonucu ve hata mesajını içerir.

preprocessing Paketi

Preprocessor, yorum satırlarını temizleyerek Parser öncesi kaynak kodu hazırlar. String literal içindeki // veya /* */ benzeri ifadelerin yanlışlıkla yorum kabul edilmemesi bu paketin önemli sorumluluğudur.

parsing Paketi

Lexer token listesi üretir; Parser bu token listesinden AST oluşturur. Token ve ASTNode sınıfları, daha sonraki analizlerin üzerinde çalışacağı temel veri yapılarını sağlar.

analysis Paketi

SyntaxAnalyzer sözdizimi hatalarını bulur. RuleEngine, IRule arayüzünü uygulayan tüm kalite kurallarını çalıştırır. MetricCalculator ise LOC, fonksiyon sayısı, struct sayısı gibi metrikleri hesaplar.

results Paketi

Diagnostic sınıfı syntax hatası, kural ihlali veya bilgi mesajını tek tip veri modeliyle temsil eder. AnalysisResult tüm diagnostikleri ve metrikleri tek nesnede birleştirir. SeverityClassifier, bulgulara kritik, uyarı veya bilgi seviyesi atar.

reporting Paketi

ReportManager rapor oluşturma sürecini yönetir. IExporter arayüzü sayesinde TXT, JSON ve HTML dışı aktarım sınıfları aynı sözleşmeye bağlı çalışır.

ui Paketi

MainWindow ana ekranı yönetir. CodeEditor kaynak kodu gösterir ve ilgili satıra gitme/vurgulama işlemlerini yapar. DiagnosticListPanel bulguları listeler ve önem seviyesine göre filtreler. MetricPanel metrikleri özetler.

3.4 Sınıf Arayüzleri

Bu bölümde önemli sınıfların öznitelikleri ve metotları tasarım seviyesinde açıklanmıştır. Her metot için metodun imzası, aldığı parametreler, dönüş değeri ve görevi ayrı ayrı verilmiştir. Bu format, sınıf arayüzlerinin geliştirme aşamasında doğrudan referans alınabilmesini amaçlar.

Application <<control>>

Uygulamanın giriş noktasını ve genel yaşam döngüsünü yöneten üst seviye kontrol sınıfıdır.

Öznitelikler:

Öznitelik	Tür	Açıklama
mainWindow	MainWindow*	Ana pencere nesnesini tutar.
analysisEngine	AnalysisEngine	Analiz işlemlerini yürüten merkezi servis nesnesini tutar.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void start()	Yok	void	Uygulamayı başlatır, gerekli arayüz ve analiz bileşenlerini oluşturur.

void shutdown()	Yok	void	Uygulama kapatılırken açık kaynakları serbest bırakır.
-----------------	-----	------	--

AnalysisEngine <<control>>

Dosya alındıktan sonra ön işleme, ayrıştırma, analiz, sınıflandırma ve sonuç üretme adımlarını yöneten merkezi orkestratördür.

Öznitelikler:

Öznitelik	Tür	Açıklama
fileLoader	FileLoader	Dosya yükleme bileşeni.
preprocessor	Preprocessor	Yorum temizleme bileşeni.
lexer	Lexer	Token üretme bileşeni.
parser	Parser	AST oluşturma bileşeni.
syntaxAnalyzer	SyntaxAnalyzer	Syntax hata analizi bileşeni.
ruleEngine	RuleEngine	Kod kalitesi kurallarını çalıştıran bileşen.
metricCalculator	MetricCalculator	Kod metriklerini hesaplayan bileşen.
severityClassifier	SeverityClassifier	Bulgulara önem seviyesi atayan bileşen.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
AnalysisResult analyze(SourceFile file)	file: Analiz edilecek SourceFile nesnesi	AnalysisResult	Tüm analiz pipeline adımlarını çalıştırır ve tek bir AnalysisResult döndürür.
bool isReady()	Yok	bool	Analiz motorundaki gerekli alt bileşenlerin hazır olup olmadığını kontrol eder.
void reset()	Yok	void	Önceki analizden kalan geçici verileri temizler.

SourceFile <<entity>>

Kullanıcının sisteme yüklediği C kaynak kod dosyasını temsil eden veri sınıfıdır.

Öznitelikler:

Öznitelik	Tür	Açıklama
-----------	-----	----------

path	string	Dosyanın tam yolu.
fileName	string	Dosyanın adı.
content	string	Dosyanın metin içeriği.
size	long	Dosya boyutu.
loadedAt	DateTime	Dosyanın sisteme yüklendiği zaman.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
string getContent()	Yok	string	Kaynak kod içeriğini döndürür.
string getExtension()	Yok	string	Dosya uzantısını döndürür.
bool isEmpty()	Yok	bool	Dosya içeriğinin boş olup olmadığını kontrol eder.
long getSize()	Yok	long	Dosya boyutunu döndürür.

FileLoader <<control>>

Diskten dosya okuma işlemini gerçekleştirir ve geçerli dosyayı SourceFile nesnesine dönüştürür.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
SourceFile load(string path)	path: Okunacak dosyanın yolu	SourceFile	Belirtilen dosyayı okur ve SourceFile nesnesi olarak döndürür.
bool exists(string path)	path: Kontrol edilecek dosya yolu	bool	Dosyanın fiziksel olarak var olup olmadığını kontrol eder.
string readText(string path)	path: Okunacak dosya yolu	string	Dosyanın metin içeriğini döndürür.

FileValidator <<control>>

Yüklenen dosyanın analiz için uygun olup olmadığını kontrol eder.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
ValidationResult validate(SourceFile file)	file: Doğrulanacak kaynak dosya	ValidationResult	Uzantı, boşluk, okunabilirlik ve binary dosya kontrollerinin sonucunu döndürür.
bool hasValidExtension(SourceFile file)	file: Kontrol edilecek dosya	bool	Dosyanın .c uzantılı olup olmadığını kontrol eder.
bool isReadable(SourceFile file)	file: Kontrol edilecek dosya	bool	Dosyanın okunabilir olup olmadığını kontrol eder.
bool isBinary(SourceFile file)	file: Kontrol edilecek dosya	bool	Dosyanın binary formatta olup olmadığını kontrol eder.

ValidationResult <<entity>>

Dosya doğrulama işleminin sonucunu ve hata mesajını taşıyan hafif veri sınıfıdır.

Öznitelikler:

Öznitelik	Tür	Açıklama
valid	bool	Doğrulama sonucu.
message	string	Başarısızlık veya uyarı mesajı.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
bool isValid()	Yok	bool	Doğrulama başarılıysa true döndürür.
string getMessage()	Yok	string	Doğrulama sonucuna ait mesajı döndürür.

Preprocessor <<control>>

Kaynak kodu analiz öncesi temizleyen bileşendir. Yorum satırlarını analiz dışı bırakır ve satır numarası bütünlüğünü korur.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
string process(string code)	code: Ham kaynak kod	string	Yorumları temizlenmiş kaynak kodu döndürür.
string removeSingleLineComments(string code)	code: Ham veya ara kaynak kod	string	// ile başlayan tek satırlı yorumları temizler.
string removeMultiLineComments(string code)	code: Ham veya ara kaynak kod	string	/* */ biçimindeki çok satırlı yorumları temizler.
bool isInsideStringLiteral(int index, string code)	index: Kontrol edilecek karakter konumu; code: Kaynak kod	bool	Belirtilen konumun string literal içinde olup olmadığını kontrol eder.

Lexer <<control>>

Kaynak kodu C dilinin anlamlı birimlerine, yani tokenlara ayıran sözlüksel analiz bileşenidir.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
vector<Token> tokenize(string code)	code: Ön işlenmiş kaynak kod	vector<Token>	Kaynak koddan üretilen token listesini döndürür.
Token nextToken()	Yok	Token	Sıradaki tokeni üretir.
bool isKeyword(string value)	value: Kontrol edilecek kelime	bool	Kelimenin C anahtar kelimesi olup olmadığını kontrol eder.

Token <<entity>>

Lexer tarafından üretilen tek bir anlamlı kod birimini temsil eder.

Öznitelikler:

Öznitelik	Tür	Açıklama
type	TokenType	Token türü.
value	string	Tokenın metinsel değeri.

line	int	Tokenın bulunduğu satır.
column	int	Tokenın bulunduğu sütun.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
TokenType getType()	Yok	TokenType	Token türünü döndürür.
string getValue()	Yok	string	Token metin değerini döndürür.
int getLine()	Yok	int	Tokenın satır numarasını döndürür.
int getColumn()	Yok	int	Tokenın sütun numarasını döndürür.

Parser <<control>>

Token listesinden AST oluşturan sözdizimsel analiz bileşenidir.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
ASTNode parse(vector<Token> tokens)	tokens: Lexer tarafından üretilen token listesi	ASTNode	Token listesinden program kökünü temsil eden ASTNode döndürür.
ASTNode parseFunction()	Yok	ASTNode	Fonksiyon tanımını ayrıştırır.
ASTNode parseStatement()	Yok	ASTNode	Bir C deyimini ayrıştırır.
vector<Diagnostic> getParseDiagnostics()	Yok	vector<Diagnostic>	Parsing sırasında oluşan hataları döndürür.

ASTNode <<entity>>

Soyut Sözdizim Ağacındaki tek bir düğümü temsil eder.

Öznitelikler:

Öznitelik	Tür	Açıklama
type	ASTNodeType	Düğüm türü.
value	string	Düğüm değeri.

line	int	Düğümün kaynak koddaki satırı.
children	vector<ASTNode>	Alt düğümler.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void addChild(ASTNode child)	child: Eklenecek alt düğüm	void	AST düğümüne yeni alt düğüm ekler.
vector<ASTNode> getChildren()	Yok	vector<ASTNode>	Alt düğüm listesini döndürür.
ASTNodeType getType()	Yok	ASTNodeType	Düğüm türünü döndürür.
int getLine()	Yok	int	Düğümün satır numarasını döndürür.

SyntaxAnalyzer <<control>>

AST ve tokenlar üzerinden syntax hatalarını tespit eden analiz bileşenidir.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
vector<Diagnostic> analyze(ASTNode ast)	ast: Parser tarafından oluşturulan AST	vector<Diagnostic>	Eksik noktalı virgül, parantez hatası gibi syntax bulgularını döndürür.
bool checkParentheses(vector<Token> tokens)	tokens: Token listesi	bool	Yuvarlak, süslü ve köşeli parantezlerin eşleşmesini kontrol eder.
Diagnostic createSyntaxDiagnostic(int line, string message)	line: Hata satırı; message: Hata açıklaması	Diagnostic	Syntax hatası için Diagnostic nesnesi oluşturur.

RuleEngine <<control>>

IRule arayüzünü uygulayan tüm kod kalitesi kurallarını AST üzerinde çalıştırır.

Öznitelikler:

Öznitelik	Tür	Açıklama
rules	vector<IRule*>	Çalıştırılacak kural nesneleri listesi.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void addRule(IRule* rule)	rule: Sisteme eklenecek kural nesnesi	void	Yeni bir analiz kuralını RuleEngine içine ekler.
vector<Diagnostic> run(ASTNode ast)	ast: Analiz edilecek AST	vector<Diagnostic>	Tüm kuralları çalıştır ve üretilen bulguları tek listede döndürür.
int getRuleCount()	Yok	int	Kayıtlı kural sayısını döndürür.
void clearRules()	Yok	void	Kayıtlı kuralları temizler.

IRule <<interface>>

Kod kalitesi kurallarının uygulaması gereken ortak arayüzdür. Strategy Pattern bu arayüz üzerinden uygulanır.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
string getId()	Yok	string	Kural kimliğini döndürür. Örnek: R001.
string getName()	Yok	string	Kural adını döndürür.
DiagnosticSeverity getDefaultSeverity()	Yok	DiagnosticSeverity	Kural ihlalinin varsayılan önem seviyesini döndürür.
vector<Diagnostic> check(ASTNode ast)	ast: Üzerinde denetim yapılacak AST	vector<Diagnostic>	Kuralı AST üzerinde çalıştırır ve ihlal listesini döndürür.

Somut Kural Sınıfları <<entity/control>>

Aşağıdaki sınıflar IRule arayüzünü uygular. Her biri aynı temel metot sözleşmesini kullanır; farklılık, check(ASTNode ast) metodunda yürütülen kural mantığındadır.

Sınıf	Temel Metot	Parametre	Dönüş Değeri	Açıklama
UnusedVariableRule	vector<Diagnostic> check(ASTNode ast)	ast: Değişken tanım ve kullanım bilgilerini içeren AST	vector<Diagnostic>	Tanımlanmış ancak kullanılmamış değişkenleri tespit eder.
UninitializedVarRule	vector<Diagnostic> check(ASTNode ast)	ast: Değişken atama ve kullanım akışını içeren AST	vector<Diagnostic>	İlk değer verilmeden kullanılan değişkenleri tespit eder.
MagicNumberRule	vector<Diagnostic> check(ASTNode ast)	ast: Sayısal literalleri içeren AST	vector<Diagnostic>	0 ve 1 gibi istisnalar dışında açıklamasız sayısal sabitleri raporlar.
FunctionLengthRule	vector<Diagnostic> check(ASTNode ast)	ast: Fonksiyon düğümlerini içeren AST	vector<Diagnostic>	Belirlenen maksimum satır sınırını aşan fonksiyonları tespit eder.
GotoUsageRule	vector<Diagnostic> check(ASTNode ast)	ast: Deyim düğümlerini içeren AST	vector<Diagnostic>	goto kullanımını uyarı olarak raporlar.
InfiniteLoopRule	vector<Diagnostic> check(ASTNode ast)	ast: Döngü düğümlerini içeren AST	vector<Diagnostic>	Potansiyel sonsuz döngü yapılarını tespit eder.

MetricCalculator <<control>>

AST ve kaynak kod üzerinde nicel metrik hesaplamaları yapan bileşendir.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
vector<CodeMetric> calculate(ASTNode ast, string code)	ast: Kod yapısı; code: Kaynak kod metni	vector<CodeMetric>	LOC, fonksiyon sayısı, struct sayısı gibi metrikleri döndürür.
int countLines(string code)	code: Kaynak kod metni	int	Toplam satır sayısını hesaplar.
int countFunctions(ASTNode ast)	ast: AST veri yapısı	int	Fonksiyon tanımı sayısını hesaplar.
int countStructs(ASTNode ast)	ast: AST veri yapısı	int	Struct tanımı sayısını hesaplar.

Diagnostic <<entity>>

Syntax hatası, kural ihlali veya bilgi mesajını tek tip modelleyen bulgu sınıfıdır.

Öznitelikler:

Öznitelik	Tür	Açıklama
line	int	Bulgunun ilgili olduğu satır numarası.
message	string	Kullanıcıya gösterilecek açıklama.
severity	DiagnosticSeverity	Kritik, uyarı veya bilgi seviyesi.
source	string	Bulgunun kaynağı: syntax veya rule.
ruleId	string	Kural kaynaklı bulgularda kural kimliği.
codeSnippet	string	İlgili kod parçası.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
--------------	--------------	--------------	----------

int getLine()	Yok	int	Bulgunun satır numarasını döndürür.
string getMessage()	Yok	string	Bulgu açıklamasını döndürür.
DiagnosticSeverity getSeverity()	Yok	DiagnosticSeverity	Bulgunun önem seviyesini döndürür.
string getCodeSnippet()	Yok	string	İlgili kod parçasını döndürür.

CodeMetric <<entity>>

Tek bir kod metriğini ad, değer ve birim bilgisiyle temsil eder.

Öznitelikler:

Öznitelik	Tür	Açıklama
name	string	Metrik adı. Örnek: LOC.
value	double	Metrik değeri.
unit	string	Metrik birimi. Örnek: satır, adet.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
string getName()	Yok	string	Metrik adını döndürür.
double getValue()	Yok	double	Metrik değerini döndürür.
string getUnit()	Yok	string	Metrik birimini döndürür.

SeverityClassifier <<control>>

Bulgulara türlerine göre önem seviyesi ve görsel renk karşılığı atar.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
DiagnosticSeverity classify(Diagnostic diagnostic)	diagnostic: Seviyesi belirlenecek bulgu	DiagnosticSeverity	Bulgunun kritik, uyarı veya bilgi seviyesini döndürür.

string getColor(DiagnosticSeverity severity)	severity: Önem seviyesi	string	Arayüzde kullanılacak renk kodunu döndürür.
vector<Diagnostic> classifyAll(vector<Diagnostic> c> diagnostics)	diagnostics: Sınıflandırılacak bulgu listesi	vector<Diagnostic>	Tüm bulgulara önem seviyesi uygulanmış listeyi döndürür.

AnalysisResult <<entity>>

Bir analiz çalışmasının tüm çıktısını tutan ana sonuç nesnesidir.

Öznitelikler:

Öznitelik	Tür	Açıklama
diagnostics	vector<Diagnostic>	Syntax ve kural bulguları.
metrics	vector<CodeMetric>	Hesaplanan metrikler.
fileInfo	SourceFile	Analiz edilen dosya bilgisi.
analysisTimeMs	long	Analizin milisaniye cinsinden süresi.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
vector<Diagnostic> getDiagnostics()	Yok	vector<Diagnostic>	Tüm bulguları döndürür.
vector<Diagnostic> getDiagnosticsBySeverity(DiagnosticSeverity level)	level: Filtrelenecek önem seviyesi	vector<Diagnostic>	Belirli önem seviyesindeki bulguları döndürür.
vector<CodeMetric> getMetrics()	Yok	vector<CodeMetric>	Hesaplanan metrikleri döndürür.
int getCriticalCount()	Yok	int	Kritik seviyeli bulgu sayısını döndürür.

bool hasDiagnostics()	Yok	bool	Herhangi bir bulgu olup olmadığını kontrol eder.
-----------------------	-----	------	--

ReportManager <<control>>

Analiz sonucunu kullanıcıya sunulacak rapor formatına dönüştürür ve dışa aktarımı koordine eder.

Öznitelikler:

Öznitelik	Tür	Açıklama
exporters	vector<IExporter*>	Desteklenen rapor dışa aktarıcıları listesi.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void registerExporter(IExporter* exporter)	exporter: Eklenecek dışa aktarıcı nesne	void	Yeni bir rapor formatını sisteme kaydeder.
void generateReport(AnalysisResult result)	result: Raporlanacak analiz sonucu	void	Analiz sonucundan rapor içeriği üretir.
void exportResult(string format, AnalysisResult result, string path)	format: txt/json/html; result: Analiz sonucu; path: Kayıt yolu	void	Seçilen formata uygun exporter nesnesini çağırarak raporu kaydeder.
IExporter* findExporter(string format)	format: İstenen rapor formatı	IExporter*	İstenen formata uygun dışa aktarıcı nesneyi döndürür.

IExporter <<interface>>

Rapor dışa aktarım formatlarının uygulaması gereken ortak arayüzdür.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
--------------	--------------	--------------	----------

string getFormatName()	Yok	string	Exporter tarafından desteklenen format adını döndürür.
void exportResult(AnalysisResult result, string path)	result: Dışa aktarılacak analiz sonucu; path: Hedef dosya yolu	void	Analiz sonucunu ilgili formatta diske yazar.

Somut Exporter Sınıfları <<control>>

Sınıf	Metot	Parametreler	Dönüş Değeri	Açıklama
TxtExporter	void exportResult(AnalysisResult result, string path)	result: Analiz sonucu; path: .txt dosya yolu	void	Sonuçları okunabilir düz metin raporu olarak kaydeder.
JsonExporter	void exportResult(AnalysisResult result, string path)	result: Analiz sonucu; path: .json dosya yolu	void	Sonuçları makine tarafından işlenebilir JSON formatında kaydeder.
HtmlExporter	void exportResult(AnalysisResult result, string path)	result: Analiz sonucu; path: .html dosya yolu	void	Sonuçları tarayıcıda açılabilir HTML raporu olarak kaydeder.

MainWindow <<boundary>>

Kullanıcının sistemle etkileşim kurduğu ana pencere bileşenidir.

Öznitelikler:

Öznitelik	Tür	Açıklama
codeEditor	CodeEditor	Kaynak kod görüntüleme alanı.
diagnosticPanel	DiagnosticListPanel	Bulgu listeleme paneli.
metricPanel	MetricPanel	Metrik gösterim paneli.
toolbar	Toolbar	Komut butonları.
statusBar	StatusBar	Durum ve ilerleme bilgisi.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void openFile()	Yok	void	Kullanıcıdan .c dosyası seçmesini ister ve dosyayı yükleme sürecini başlatır.
void startAnalysis()	Yok	void	Yüklenen dosya için AnalysisEngine.analyze() çağrısını başlatır.
void showResults(AnalysisResult result)	result: Gösterilecek analiz sonucu	void	Bulguları ve metrikleri ilgili arayüz panellerine dağıtır.
void showError(string message)	message: Kullanıcıya gösterilecek hata metni	void	Hata mesajını arayüzde gösterir.

CodeEditor <<boundary>>

Kaynak kodun görüntülendiği ve bulgu satırlarına navigasyon yapılan arayüz bileşenidir.

Öznitelikler:

Öznitelik	Tür	Açıklama
currentCode	string	Editörde gösterilen kaynak kod.
highlightedLine	int	Vurgulanan satır numarası.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void setCode(string code)	code: Gösterilecek kaynak kod	void	Editör alanına kaynak kodu yerleştirir.
void goToLine(int line)	line: Gidilecek satır numarası	void	Editörü ilgili satıra kaydırır.
void highlight(int line)	line: Vurgulanacak satır numarası	void	Belirtilen satırı görsel olarak vurgular.
void clearHighlights()	Yok	void	Önceki vurgulamaları temizler.

DiagnosticListPanel <<boundary>>

Analiz bulgularını listeleyen ve filtreleme sağlayan arayüz bileşenidir.

Öznitelikler:

Öznitelik	Tür	Açıklama
diagnostics	vector<Diagnostic>	Panelde gösterilen bulgu listesi.

activeFilter	DiagnosticSeverity	Geçerli önem seviyesi filtresi.
--------------	--------------------	---------------------------------

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void display(vector<Diagnostic> list)	list: Gösterilecek bulgu listesi	void	Bulguları satır, mesaj ve önem seviyesiyle listeler.
void filterBySeverity(DiagnosticSeverity level)	level: Kritik/uyarı/bilgi filtresi	void	Listeyi seçilen önem seviyesine göre filtreler.
Diagnostic getSelectedDiagnostic()	Yok	Diagnostic	Kullanıcının seçtiği bulguyu döndürür.
void clear()	Yok	void	Paneldeki bulgu listesini temizler.

MetricPanel <<boundary>>

Kod metriklerini kullanıcıya özet biçimde gösteren arayüz bileşenidir.

Öznitelikler:

Öznitelik	Tür	Açıklama
metrics	vector<CodeMetric>	Gösterilen metrik listesi.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void displayMetrics(vector<CodeMetric> metrics)	metrics: Gösterilecek metrikler	void	LOC, fonksiyon sayısı, struct sayısı gibi metrikleri panelde gösterir.
void clear()	Yok	void	Metrik panelini temizler.

Toolbar <<boundary>>

Kullanıcının dosya açma, analiz başlatma ve rapor alma gibi temel işlemleri tetiklediği arayüz bileşenidir.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void onOpenFileClicked()	Yok	void	Dosya açma olayını MainWindow.openFile() ile ilişkilendirir.
void onAnalyzeClicked()	Yok	void	Analiz başlatma olayını MainWindow.startAnalysis() ile ilişkilendirir.
void onExportClicked()	Yok	void	Rapor dışa aktarma sürecini başlatır.
void setAnalyzeEnabled(bool enabled)	enabled: Buton aktiflik bilgisi	void	Geçerli dosya yoksa analiz butonunu pasif hale getirir.

StatusBar <<boundary>>

Kullanıcıya anlık durum ve ilerleme bilgisi gösteren alt çubuk bileşenidir.

Öznitelikler: Bu sınıfın tasarım seviyesinde zorunlu kalıcı özniteliği bulunmamaktadır veya öznitelikler uygulama detayına bırakılmıştır.

Metot Arayüzleri:

Metot İmzası	Parametreler	Dönüş Değeri	Açıklama
void showMessage(string message)	message: Gösterilecek durum mesajı	void	Kullanıcıya kısa durum bilgisi gösterir.
void showProgress(int percent)	percent: 0-100 arası ilerleme oranı	void	Analiz ilerlemesini yüzdeler olarak gösterir.
void clear()	Yok	void	Durum mesajını temizler.

Enum Sınıfları ve Değerleri

Enum	Değerler	Kullanım Amacı
------	----------	----------------

TokenType	KEYWORD, IDENTIFIER, INT_LITERAL, FLOAT_LITERAL, STRING_LITERAL, CHAR_LITERAL, OPERATOR, PUNCTUATION, PREPROCESSOR, UNKNOWN	Lexer tarafından üretilen tokenların türünü belirtir.
ASTNodeType	PROGRAM, FUNCTION_DEF, VARIABLE_DECL, IF_STATEMENT, WHILE_LOOP, FOR_LOOP, RETURN_STMT, EXPRESSION, FUNCTION_CALL, STRUCT_DEF, BLOCK	AST düğümlerinin yapısal türünü belirtir.
DiagnosticSeverity	CRITICAL, WARNING, INFO	Bulguların önem seviyesini ve arayüzdeki renk kodlamasını belirler.

3.4 Bölümü İçin Kısa Değerlendirme

Bu sınıf arayüzleri, analiz raporundaki kavramsal sınıfları tasarım seviyesine taşır. Özellikle metot parametrelerinin ve dönüş değerlerinin açık şekilde belirtilmesi, ekip üyeleri arasında görev paylaşımını kolaylaştırır. Örneğin RuleEngine sınıfı yalnızca IRule arayüzünü bilir; bu sayede yeni bir kural eklenirken mevcut analiz motoru değiştirilmez. Benzer şekilde ReportManager yalnızca IExporter arayüzüyle çalışır.

3.5. Tasarım Desenleri

Bu bölümde, CodeSentinel sisteminin esnekliğini, bakım kolaylığını ve genişletilebilirliğini artırmak amacıyla kullanılan yazılım tasarım desenleri detaylandırılmıştır.

3.5.1. Strategy Pattern — IRule Denetimi

Uygulandığı Sınıflar:

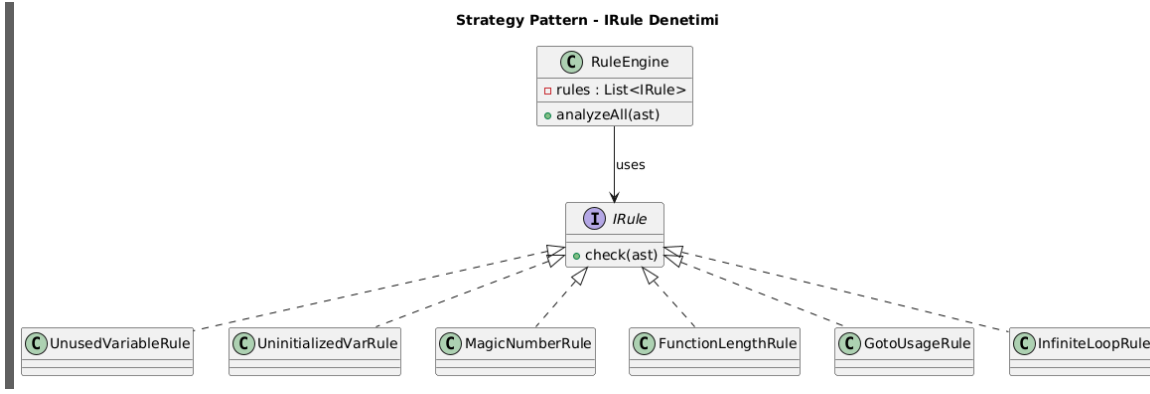
RuleEngine (Context), IRule (Arayüz), UnusedVariableRule, UninitializedVarRule, MagicNumberRule, FunctionLengthRule, GotoUsageRule, InfiniteLoopRule (Somut Stratejiler)

Tercih Gerekeçsi:

Bu tasarım deseni, sisteme yeni kod kalitesi kuralları eklenirken mevcut RuleEngine sınıfının değiştirilmesini önlemek amacıyla tercih edilmiştir. Böylece Açık/Kapalı Prensibi (Open/Closed Principle) sağlanmaktadır.

İşleyiş:

RuleEngine sınıfı, IRule arayüzünü uygulayan bir kural listesi tutmaktadır. Analiz süreci başlatıldığında analyzeAll(ast) metodu çalıştırılır ve tüm kurallar üzerinde bir döngü kurulur. Her bir kuralın check(ast) metodu polimorfik olarak çağrılarak analiz gerçekleştirilir.



Şekil 3.5.1: IRule arayüzü ve somut kural sınıflarını gösteren Strategy Pattern sınıf diyagramı

3.5.2. Strategy Pattern — IExporter (Raporlama)

Uygulandığı Sınıflar:

ReportManager (Context), IExporter (Arayüz), TxtExporter, JsonExporter, HtmlExporter (Somut Stratejiler)

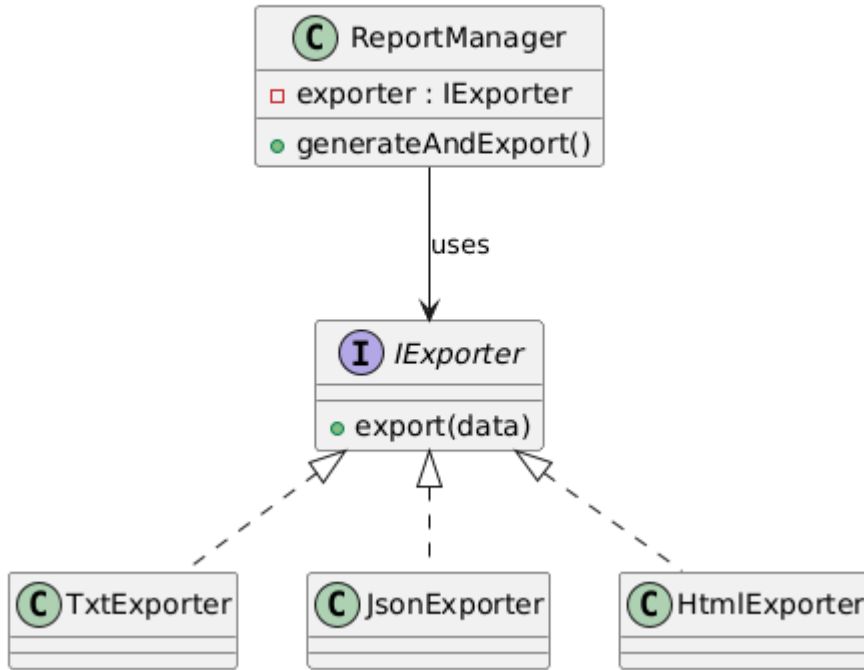
Tercih Gerekçesi:

Sistemin ilerleyen aşamalarda PDF veya CSV gibi farklı rapor formatlarını destekleyebilmesi için, mevcut raporlama yapısını değiştirmeden yeni formatların sisteme entegre edilebilmesini sağlamak amacıyla tercih edilmiştir.

İşleyiş:

ReportManager sınıfı, kullanıcı tarafından arayüz üzerinden seçilen dosya formatına göre uygun IExporter nesnesini belirler. Ardından generateAndExport() metodu çağrılarak analiz sonuçları seçilen formata dönüştürülür ve dışa aktarılır.

Strategy Pattern - IExporter



Şekil 3.5.2: IExporter arayüzü ve ilgili exporter sınıflarını gösteren Strategy Pattern sınıf diyagramı.

3.5.3. Facade Pattern — AnalysisEngine

Uygulandığı Sınıflar:

AnalysisEngine

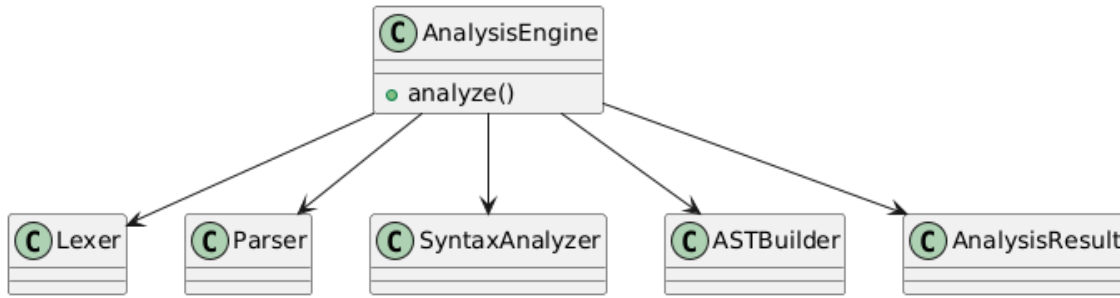
Tercih Gerekçesi:

Kullanıcı arayüzü (UI) katmanının, analiz sürecinin karmaşık iç yapısından (dosya ön işleme, tokenization, AST oluşturma vb.) izole edilmesi amacıyla kullanılmıştır. Böylece sistem daha anlaşılır ve yönetilebilir hale getirilmiştir.

İşleyiş:

Kullanıcı arayüzü, analiz sürecini başlatmak için alt bileşenlerle (Lexer, Parser, SyntaxAnalyzer vb.) doğrudan iletişim kurmak yerine yalnızca AnalysisEngine sınıfının sunduğu analyze() metodunu çağırır. AnalysisEngine, tüm alt sistemleri koordine ederek tek bir AnalysisResult nesnesi döndürür.

Facade Pattern - AnalysisEngine



Şekil 3.5.3: AnalysisEngine ve alt sistemler arasındaki ilişkiyi gösteren Facade Pattern diyagramı.

Sözlük (Glossary)

Static Code Analysis

Kaynak kodun çalıştırılmadan analiz edilerek hata ve güvenlik açıklarının tespit edilmesi sürecidir.

Abstract Syntax Tree (AST)

Kaynak kodun hiyerarşik yapısını temsil eden ağaç veri yapısıdır.

Lexer (Lexical Analyzer)

Kaynak kodu anlamlı parçalara (token) ayıran bileşendir.

Parser

Token'lerden programın yapısal modelini (AST) oluşturan bileşendir.

Token

Programlama dilindeki en küçük anlamlı birimdir (keyword, identifier vb.).

Rule Engine

Kod üzerinde tanımlı kuralları çalıştırarak hata ve ihlal tespiti yapan sistem bileşenidir.

Diagnostic

Tespit edilen hata, uyarı veya bilgi mesajını temsil eden veri yapısıdır.

Code Metric

Kodun nicel özelliklerini ölçen değerlerdir (LOC, fonksiyon sayısı vb.).

Buffer Overflow

Bir bellek alanının sınırlarının aşılmasıyla oluşan güvenlik açığıdır.

Dangling Pointer

Serbest bırakılmış belleğe işaret eden pointer durumudur.

Design Pattern

Yazılım tasarımında tekrar eden problemlere sunulan genel çözümlerdir.

Strategy Pattern

Algoritmaların değiştirilebilir şekilde kapsüllenmesini sağlayan tasarım deseni.

Facade Pattern

Karmaşık alt sistemleri tek bir arayüz üzerinden yöneten tasarım deseni.

False Positive

Gerçekte hata olmayan bir durumun hata olarak raporlanması.

Referanslar

- [1] N. Ayewah, W. Pugh, J. D. Morgenthaler, J. Penix and Y. Zhou,
"A Few Billion Lines of Code Later: Using Static Analysis to Find Bugs in the Real World,"
IEEE Software, vol. 27, no. 1, pp. 66-75, Jan.-Feb. 2010.
- [2] K. D. Cooper and L. Torczon,
Engineering a Compiler, 2nd ed., Burlington, MA, USA: Morgan Kaufmann, 2011.
- [3] A. V. Aho, M. S. Lam, R. Sethi and J. D. Ullman,
Compilers: Principles, Techniques, and Tools, 2nd ed., Boston, MA, USA: Addison-Wesley, 2006.
- [4] S. Lu, S. Park, E. Seo and Y. Zhou,
"A Study of Concurrency Bugs in Real-World Programs,"
in Proc. ACM Symposium on Operating Systems Principles (SOSP),
Stevenson, WA, USA, 2008, pp. 105-116.
- [5] S. R. Chidamber and C. F. Kemerer,
"A Metrics Suite for Object-Oriented Design,"
IEEE Transactions on Software Engineering, vol. 20, no. 6, pp. 476-493, June 1994.
- [6] N. E. Fenton and J. Bieman,
Software Metrics: A Rigorous and Practical Approach, 3rd ed., Boca Raton, FL, USA: CRC Press,
2014.